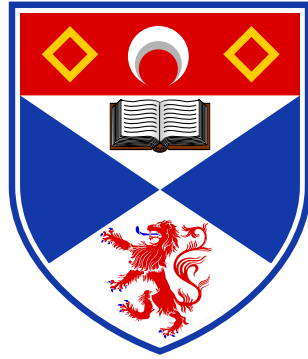




Scalable Bayesian Inference using MCMC Algorithms

Graham Heathcote

University of St Andrews

Computer Science BSc

Contents

1	Introduction	1
2	Abstract	1
3	Background	1
3.1	Distributions	2
3.2	Loss Functions	2
3.3	Logistic Regression	3
3.4	Bias Term	4
3.5	Gradient Descent	4
3.6	Likelihood Functions	4
3.7	Neural Networks	4
3.8	Markov Chains	5
3.9	Bayes Rule	5
4	Motivating Scalable Bayesian Inference	6
4.1	Understanding the posterior	6
4.2	Bayesian Occam's Razor	9
4.3	Mini Batch Algorithms	10
5	Introduction to MCMC	10
5.1	Metropolis-Hastings Algorithm	10
5.1.1	Explanation	11
5.1.2	Intuition	11
5.1.3	Example	12
5.2	Langevin MCMC	14
5.3	Naive Stochastic MCMC	15
5.4	Stochastic Gradient Langevin Dynamics	15
5.4.1	Example	16
6	Motivating Hamiltonian Dynamics	17
6.1	High Dimensional Spaces	18
6.2	MCMC Performance in High Dimensions	19
7	Hamiltonian Dynamics	20
7.1	Intuition	20
7.2	Hamiltonian Dynamics for Optimisation	21
7.3	Hamiltonian Monte Carlo	22
7.3.1	Exploring the Typical Set with HMC	23
7.4	Stochastic Gradient Hamiltonian Monte Carlo	24
8	Results	24
8.1	Two Moon Dataset Analysis	25
8.2	MNIST Dataset Analysis	26
9	Future Work	28
A	Comparison of Iterators	29
B	Proof of the Transition Kernel	29
C	Proof of Detailed Balance	30

1 Introduction

In recent years, Machine Learning has been applied to increasingly difficult problems with increasingly large datasets [Dritsas and Trigka, 2025]. This presents two problems.

First, difficult problems rarely have a singular solution. Consider a scenario where there are three possible models, each making a unique prediction. Each model also has a probability of being the “correct” model.

Model A (34%) : Market price becomes 105%

Model B (33%) : Market price becomes 50%

Model C (33%) : Market price becomes 40%

An algorithm designed to find the model most likely to be correct would use model A to predict an increase in market prices.

An alternative strategy to predict the market would be to acknowledge that no single prediction will ever perfectly capture tomorrow’s prices. Instead, all models should predict tomorrow’s price independently of one another. To make an aggregate guess of tomorrow’s price, one can take an average of all models’ predictions, weighting the average by the probability of each model being correct.

Here, our aggregate prediction would be

$$(1.05 \cdot 0.34) + (0.50 \cdot 0.33) + (0.40 \cdot 0.33) = 0.654.$$

This corresponds to a 35% fall in prices. This example highlights why relying only on the most “correct” model can lead to worse predictions. To adopt this weighted average strategy, we must know the probability of each model being “correct”. This necessitates Bayesian machine learning.

The second problem caused by large datasets is the immense amount of computational power required to make predictions. For example, a large language model cannot conceivably use every sentence ever written to generate an output. Clearly, for certain datasets, only a small subset of data is usable for each training and inference. This reality is important when designing any machine learning algorithm.

Markov Chain Monte Carlo (MCMC) algorithms can allow us to approximate the probability of all models being the “correct” one. In recent years, new techniques have been developed to run MCMC algorithms using only a small subset of available data. This opens the door for the Bayesian approach to be applied to large datasets and to solve deep learning problems.

2 Abstract

Bayesian inference offers advantages over frequentist approaches but is computationally expensive, particularly for large datasets. MCMC algorithms address this by approximating the posterior distribution, and recent techniques allow them to operate on mini-batches of data. I provide a motivation for Bayesian inference and scalable Bayesian inference in particular. I then focus on several MCMC algorithms in detail. For each algorithm, I provide an intuition for the algorithm’s design and use examples to explain how it works. I show that more advanced techniques, including Hamiltonian Dynamics, can be used in MCMC algorithms. I show that MCMC algorithms are broadly well suited for problems with noisy data where the posterior distribution is not tightly concentrated around any single weight. To show this, I evaluate the performance of the algorithms, then reduce the size of the training dataset to make the data noisier and compare again.

3 Background

This section provides a brief background in the statistics needed to understand my work. For more information, I found the first two chapters of Pattern Recognition and Machine Learning by Christopher Bishop [Bishop, 2006] very helpful. Further information on Markov Chains can be found in [Norris].

3.1 Distributions

A continuous random variable can take on infinite possible values. We can represent the probability of each of these possibilities with a function $f(x)$ such that

$$\int_{-\infty}^{\infty} f(x) dx = 1.$$

One such function that can represent a continuous random variable is a Gaussian distribution. We call this function a distribution since it represents the probabilities of all possible values for x . A Gaussian distribution, also known as a Normal distribution, is given by

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(\frac{-(x - \mu)^2}{2\sigma^2}\right)$$

where μ represents the most likely value, or the mean, and σ^2 represents the variance. Increasing the variance means a wider distribution where the “probability mass” is spread among more possible values for x .

In machine learning, we often work with high-dimensional data represented as vectors. A natural choice for modelling the probability distribution of such vectors is the multivariate Gaussian distribution:

$$f(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^k |\boldsymbol{\Sigma}|}} \cdot \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

Here, $\boldsymbol{\Sigma}$ is a covariance matrix where each element $\boldsymbol{\Sigma}_{ij}$ represents the covariance between dimensions i and j . The determinant $|\boldsymbol{\Sigma}|$ normalises the distribution to ensure it integrates to one. Note that $\mathbf{x}$ is bold to denote it being a vector of k dimensions. $\boldsymbol{\mu}$ is also a vector denoting the mean value of each element in $\mathbf{x}$.

3.2 Loss Functions

In this dissertation, I will focus on using MCMC to perform Bayesian inference in supervised machine learning models.

In supervised machine learning, a model is trained on labelled data, meaning each input corresponds to a known output. The output can be either a class (e.g., logistic regression) or a scalar (e.g., linear regression). Let us consider an example of linear regression with an input of d features and n data points.

For each data point i , we try to predict the output y_i based on input $\mathbf{x}_i$ where

$$\begin{aligned} \mathbf{x}_i &= (x_{i1}, x_{i2}, \dots, x_{id}) \\ y_i &\in \mathbb{R} \end{aligned}$$

We can make predictions by taking the product of row vector $\mathbf{x}_i$ with a column vector called θ . θ describes the model’s learned parameters. These parameters are “learned” since they are unknown initially but can be learned using an algorithm. We will discuss how to derive them later. For now, we can predict $\hat{y}_i$ based on $\mathbf{x}_i$ and θ .

$$\hat{y}_i = x_{i1} \cdot \theta_1 + x_{i2} \cdot \theta_2 + x_{i3} \cdot \theta_3 + \dots + x_{id} \cdot \theta_d$$

This can be written more concisely as

$$\hat{y}_i = \mathbf{x}_i \theta$$

Imagine we have n data points. Instead of imagining only the i th data point we can define a column vector $\mathbf{y}$ that stores y_i for all i . We can also define a matrix $\mathbf{X}$ that stores each row vector $\mathbf{x}_i$. Our matrix $\mathbf{X}$ will have d columns and n rows. $\mathbf{y}$ is a column vector with n rows.

Now, we can predict $\hat{y}_i$ for all i as follows:

$$\hat{\mathbf{y}} = \begin{bmatrix} \mathbf{x}_1 \theta \\ \mathbf{x}_2 \theta \\ \mathbf{x}_3 \theta \\ \dots \\ \mathbf{x}_n \theta \end{bmatrix}$$

This can be written more concisely using our matrix $\mathbf{X}$:

$$\hat{\mathbf{y}} = \mathbf{X} \theta$$

Therefore we can calculate $\hat{\mathbf{y}}$ given θ . The issue then becomes that we do not know θ . However, we do know $\hat{\mathbf{y}}$, our prediction based on θ , and $\mathbf{y}$, our true output. By measuring the difference between the two, we can evaluate the quality of any given θ . If θ is good, then generally $\hat{\mathbf{y}} \approx \mathbf{y}$. We thus use the loss function ℓ to measure the similarity of $\hat{\mathbf{y}}$ and $\mathbf{y}$. We can define ℓ as follows:

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

For example, if

$$\hat{\mathbf{y}} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix}$$

then

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{3} ((1-2)^2 + (2-2)^2 + (3-2)^2) = \frac{2}{3}$$

We now have a function ℓ that we can concretely use to measure the performance of θ . We could define our loss function differently, but this choice of loss function is known as mean squared error. It has this name because we square the difference between $\hat{y}_i$ and y_i , then take the mean of those squares.

3.3 Logistic Regression

For logistic regression problems where we are trying to predict whether a data point belongs to a class, we take our prediction for a data point $\mathbf{x}_i \theta$ and feed it into the logistic, or sigmoid, function given by

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

This squeezes the output between zero and one, where a value close to one represents a high probability of the data point belonging to the class, and a value close to zero represents a low probability.

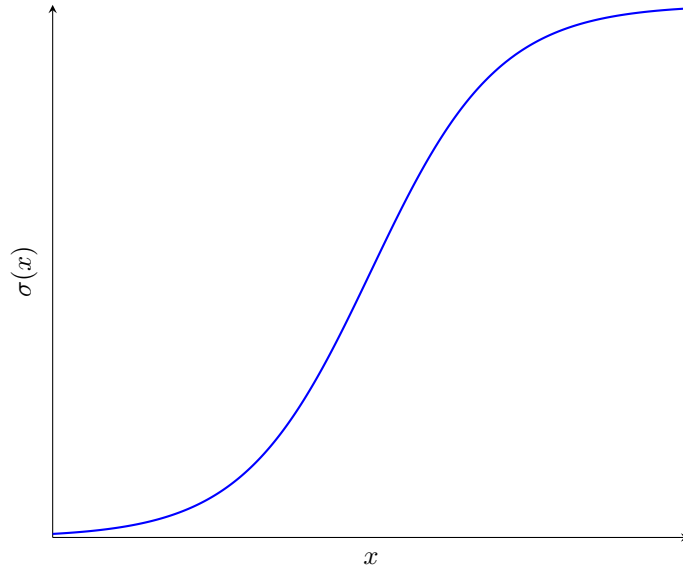


Figure 1: Sigmoid function.

This shape is often referred to as an S-curve since graphing the sigmoid function resembles an S.

Using this squeezed output, we then measure the loss of our prediction using an alternative loss function designed for classification problems:

$$\ell(\hat{y}_i, y_i) = -(y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

where $y_i \in \{0, 1\}$ is the true label and $\hat{y}_i = \sigma(\mathbf{x}_i \theta)$. This loss is low when the predicted probability $\hat{y}_i$ is close to the true label y_i , and high when they disagree. This loss function is known as cross-entropy loss. To find the cross-entropy loss across all data points, we can simply sum the individual losses and divide by the number of data points.

3.4 Bias Term

In our linear regression model, we predict $\hat{y}_i = \mathbf{x}_i\theta$. This forces our prediction to be zero when $\mathbf{x}_i$ contains only zeros. This is often unrealistic. To fix this, we add a bias term b such that

$$\hat{y}_i = \mathbf{x}_i\theta + b.$$

Having to multiply and then add makes making predictions a bit more tedious. Rather than treating b as a separate parameter, we can absorb it into θ by appending a constant 1 to each input vector $\mathbf{x}_i$:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id}, 1).$$

The corresponding extra element in θ then acts as the bias. This keeps our notation clean, since we can just write

$$\hat{\mathbf{y}} = \mathbf{X}\theta.$$

This is the same dot product as before. The only difference is that $\mathbf{X}$ now has an additional column of ones and θ now has length $d + 1$ rather than just d .

3.5 Gradient Descent

To find the optimal θ that minimizes our loss function $\ell(\theta)$, we use gradient descent. The gradient $\nabla\ell(\theta)$ points in the direction of steepest increase in loss. We iteratively update θ moving in the opposite direction

$$\theta_{t+1} = \theta_t - \alpha\nabla\ell(\theta_t)$$

where α is the learning rate, which controls the step size. By repeating this update, θ converges toward a local minimum of the loss function. We hope to reach the global minimum of the loss, as the θ at this point will maximise the similarity between $\hat{\mathbf{y}}$ and $\mathbf{y}$. Assuming we do find the global minimum, we refer to this θ value as θ_{MLE} .

An improvement on gradient descent is stochastic gradient descent. This uses only a small subset of the data, say 5%, to calculate the gradient of the loss. This gradient is then scaled up by a factor of $\frac{1}{0.05}$ to represent the full dataset. This is a simple “mini-batch algorithm” because it uses a small batch of all available data points to approximate the full dataset.

3.6 Likelihood Functions

MLE in θ_{MLE} stands for maximum likelihood estimate, since of all possible θ values, this θ is the one that is most likely to replicate our observed data.

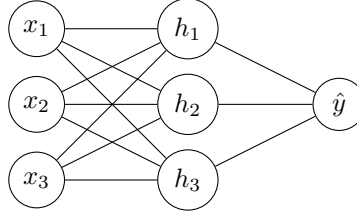
A likelihood function measures this probability for a given θ . We write this function as $p(D | \theta)$, where D represents the data and $p(D | \theta)$ represents the probability of observing it for a given θ . Maximising a likelihood function is equivalent to minimising the negative log-likelihood, which serves as a loss function. For example, in regression with Gaussian noise, the negative log-likelihood is proportional to the mean squared error (MSE).

When dealing with loss functions or likelihood functions, we often take the log of the functions. Taking the logarithm converts multiplication into addition, making calculations more numerically stable when multiplying thousands of terms together. The log does not change the outcome of gradient descent or other optimisation algorithms because log is a monotonically increasing function.

3.7 Neural Networks

The details of neural networks are not critical for this dissertation, but I have provided an intuitive overview of how they work. The important thing to know is that neural networks are used when data is not linearly separable (when patterns in the data make it impossible to cleanly divide data with a single line).

The big picture of a neural network is that the output of the first logistic regression feeds into the input of the next layer. There can be many intermediary layers, known as hidden layers, between the input and output. This can be visualised as a directed acyclic graph. Consider the following example:



In this example, there is only one hidden layer. We can describe this neural network mathematically:

$$\hat{\mathbf{y}} = \sigma(\sigma(\mathbf{XA})\mathbf{B})$$

where $\mathbf{A}$ is a matrix representing the weights between the input and hidden layers, and $\mathbf{B}$ represents the weights between the hidden and output layers.

In this example, the inner sigmoid ensures that the nodes on the hidden layer have values between zero and one. These values are then multiplied by the weight matrix $\mathbf{B}$.

Each connection has a learned weight which can be found through gradient descent. Our likelihood and loss functions therefore rely on the predictions made with all weights from all layers.

3.8 Markov Chains

A Markov Chain is a statistical system that functions similarly to a state machine. Over a time t , the system transitions from state to state according to some predefined probabilities. The probability of being in any given state only depends on the previous state.

The probability of chain X at time t being at state x can be described as

$$P(X_t = x) = \pi(x)$$

where $\pi(x)$ is a probability distribution over the random variable x .

The probability of a transition from the state at X_{t-1} to the state at X_t is described as

$$P(X_t | X_{t-1}).$$

A common property of Markov Chains is detailed balance, which is defined as follows:

$$\forall x, y \in \mathbb{X}, \pi(x) P(y | x) = \pi(y) P(x | y)$$

This property does not hold for all Markov Chains but is important in the algorithms I discuss.

Unlike state machines, Markov chains can have an infinite number of states. For example, let the domain of states be the real numbers. Detailed balance can still be established. Transitions can be defined as follows:

$$P(x | y) = \mathcal{N}(y, \sigma^2).$$

Here, the next state is sampled from a Normal distribution centred at the previous state.

3.9 Bayes Rule

Bayes' rule forms the backbone of all the MCMC algorithms I discuss. It is often defined as follows for data D and weights θ :

$$p(\theta | D) = \frac{p(D | \theta) \cdot p(\theta)}{p(D)}$$

We call $p(\theta | D)$ the posterior distribution, $p(D | \theta)$ the likelihood, $p(\theta)$ the prior distribution, and $p(D)$ the marginal likelihood, also known as the evidence or the normalising constant.

The prior distribution represents previous assumptions about θ before observing any data. This distribution is often hand-picked and centred around zero. The prior variance reflects how confident we are in the initial weights: a large variance indicates little prior certainty and allows a wide range of weights, while a small variance encodes strong confidence by concentrating mass near zero. The prior distribution does not have to be equivalent for all parameters either: one could be very confident about some elements in θ but less so about others.

The likelihood represents the probability of observing the data given θ as discussed previously. Note this is not a distribution as

$$\int_{-\infty}^{\infty} p(D | \theta) d\theta \neq 1.$$

The likelihood function is determined by the data and thus, like the data, is likely to be noisy and of a complicated shape. It is also unlikely to be narrow with a singular peak; most machine learning problems do not have a singular clear θ that when found can explain everything. Instead, there are often many possible θ values, many of which explain the data well but not perfectly.

Multiplying the prior distribution by the likelihood will not integrate to one. This presents an issue: the posterior distribution must integrate to one to be a valid distribution. We can fix this by dividing by a scalar to rescale the distribution to sum to one. This scalar, known as the marginal likelihood, can be thought of as the probability of the data D occurring given all possible θ and the probabilities of those θ values.

We can formally write this as follows:

$$p(D) = \int p(D | \theta) p(\theta) d\theta$$

The prior, $p(\theta)$, is easy to derive for any individual θ . The problem is that this integral must be taken over all possible θ , which is often impossible in high dimensional spaces. For this reason, Bayes' rule is often simplified as follows:

$$p(\theta | D) \propto p(D | \theta) p(\theta)$$

When I talk about sampling from a posterior distribution π , I am referring to this unnormalized distribution. The mode of this distribution is known as the maximum a posteriori (MAP) estimate.

4 Motivating Scalable Bayesian Inference

In this section I provide a motivation for Bayesian inference and scalable Bayesian inference in particular. I do this by contrasting the Bayesian and frequentist approaches to machine learning.

There are two main approaches to machine learning. The frequentist approach, which treats θ as an unknown yet fixed value, and the Bayesian approach, which treats θ as a random variable and seeks to understand the posterior distribution $p(\theta | D)$.

Training in a frequentist algorithm is an optimisation problem of finding θ_{MLE} via a process analogous to gradient descent. Training in a Bayesian algorithm is the process in which the posterior distribution is explored and understood. Of the two, the frequentist approach is often seen as more intuitive and is widely used in machine learning. However, it has drawbacks compared to the Bayesian method.

4.1 Understanding the posterior

In this section, I show how the Bayesian approach leads to a better understanding of the likelihood function and posterior distribution. Finally, I show how this better understanding can be leveraged to make better predictions.

Often in frequentist algorithms, little is understood about the likelihood function. Even if the likelihood function is maximised, most of the probability mass could lie far away. This results in misleading results. For example, imagine a likelihood distribution as follows:

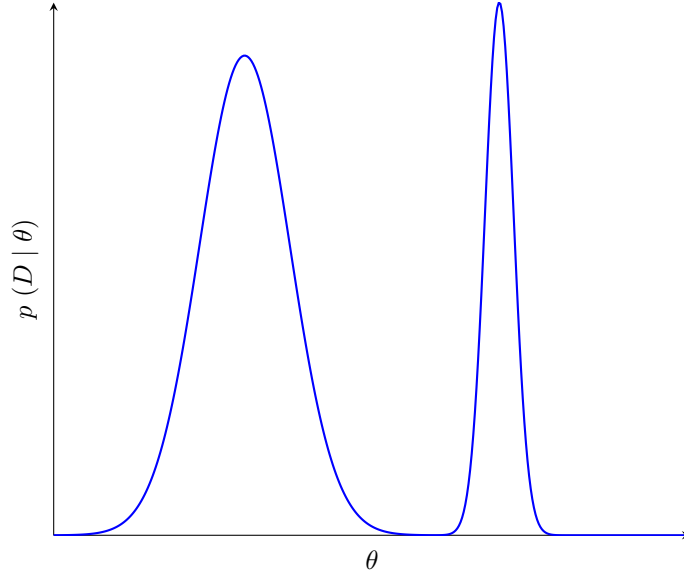


Figure 2: Example likelihood function with two peaks.

A common frequentist approach is to use θ_{MLE} , which lies at the top of the right peak, to make predictions. This point represents our single “most likely” θ , and in some cases this could be an acceptable outcome. However, even if the single most likely θ is in the right peak, more of the probability mass is in the left peak.

The Bayesian approach here is to multiply the whole distribution by our prior.

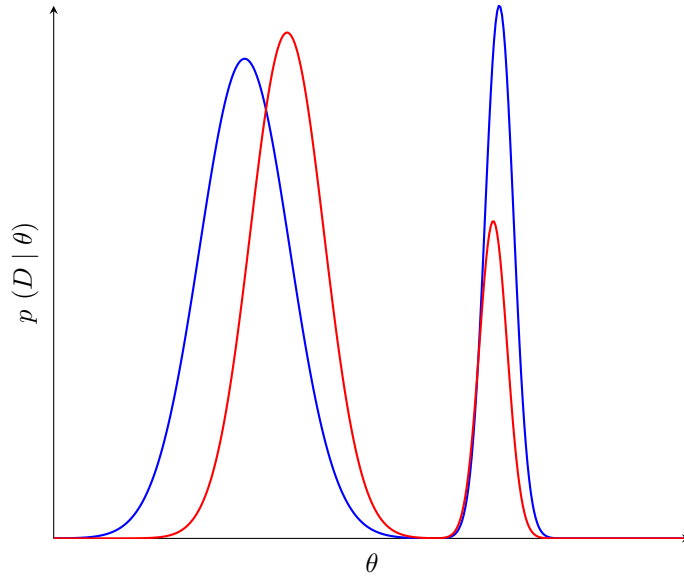


Figure 3: Example posterior function (red) overlaid on top of example likelihood function (blue).

After multiplying by the prior, our posterior is maximised in the left peak, not the right. Bayesian machine learning algorithms take this a step further. Instead of optimising to find θ_{MAP} , they approximate the full distribution. This is significant as there is no singular “correct” θ in this problem. Only the Bayesian approach would allow users to analyse the full posterior distribution and make this realisation.

Frequentists sometimes also multiply the likelihood function by the prior and optimise to find θ_{MAP} . This technique still outputs a point estimate, but in this example the estimate would at least be in the left peak. This approach is known as a “poor man’s Bayes” approximation.

To understand how we can make better predictions by understanding the full posterior rather than a point estimate, consider the following example:

Imagine we are solving a logistic regression problem where the goal is to evaluate the probability of

a point $\mathbf{x}_i$ belonging to one of two classes.

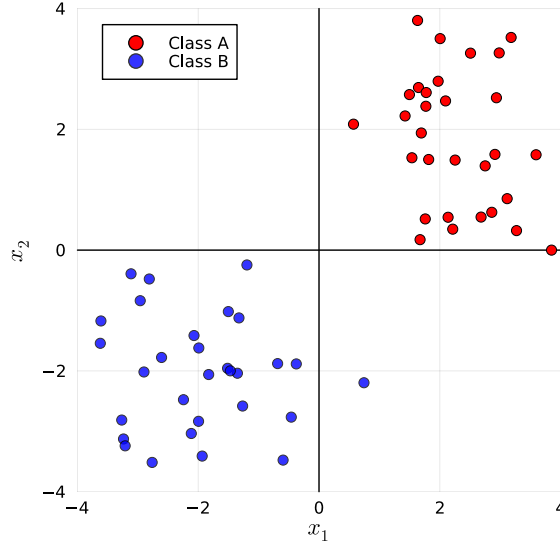


Figure 4: Example data where blue dots denote points belonging to class B and red dots belong to class A. Each dot has two features, represented by the axes.

We can write this as

$$p(y_i = A \mid D, \mathbf{x}_i)$$

where $\mathbf{x}_i$ represents the coordinates of the new data point we are trying to classify, D represents the observed data, and $p(y_i = A \mid D, \mathbf{x}_i)$ represents the probability of our new point belonging to class A. The unknowns here are our weights θ . The frequentist approach is to use the most likely weights to make predictions, i.e. $p(y_i = A \mid \theta_{MLE}, \mathbf{x}_i)$. We can visualise how this approach classifies points by generating a contour of $p(y_i = A \mid \theta_{MLE}, \mathbf{x}_i)$ for all possible $\mathbf{x}_i$.

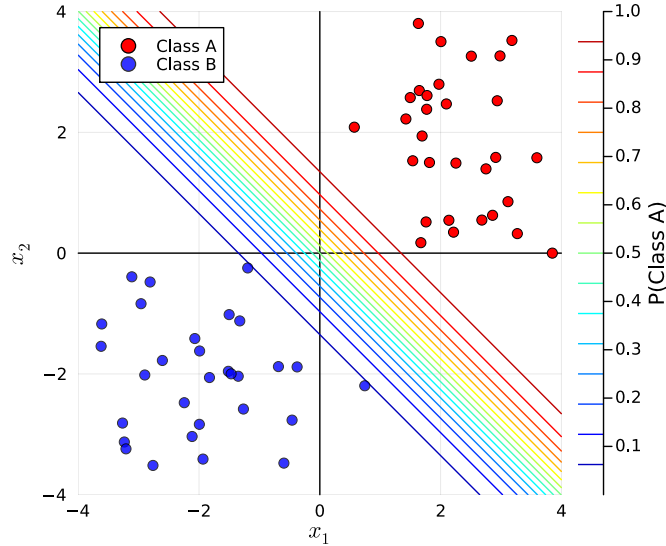


Figure 5: Class predictions using θ_{MLE} to make predictions.

For all $\mathbf{x}$, predictions are made using $\sigma(\mathbf{x}\theta_{MLE})$. Since $\mathbf{x}\theta_{MLE}$ is a linear operation, the result is a plane. If we take the sigmoid of each value on the plane the result is an S-curve.

To improve on this, we can use the Bayesian approach. Instead of considering any singular θ , let us take the expectation

$$p(Y = 1 \mid D, \mathbf{x}_i) = \int p(Y = 1 \mid \theta, \mathbf{x}_i) \cdot p(\theta \mid D) d\theta.$$

This expectation does not have a closed form solution, but we can use MCMC to approximate it. MCMC gives us an approximation of the posterior distribution $p(\theta | D)$. Predictions are made by finding $\sigma(\mathbf{x}\theta)$ and weighing each prediction by the value of the posterior at that θ .

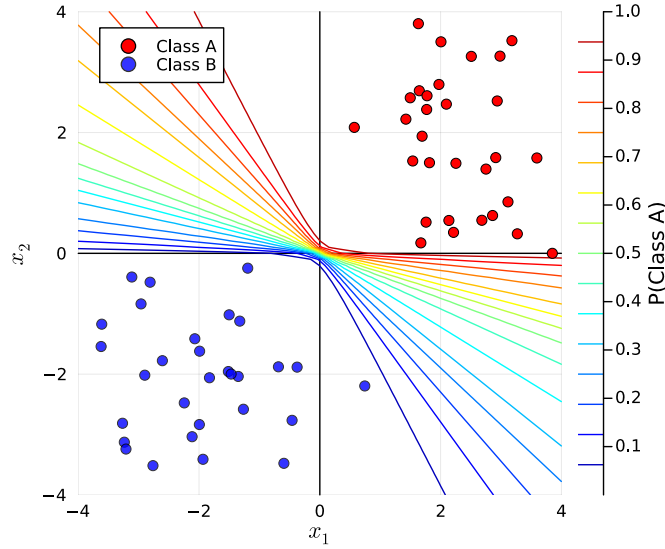


Figure 6: Class predictions using MCMC to make predictions.

The resulting contour is the sum of many S-curves representing different values the posterior can take on. Comparing the contours, the difference between treating θ as a single value versus a random variable is clear. Imagine two data points:

$$x_a = \{-2, 0\}, x_b = \{-4, 2\}$$

The frequentist contour would show these points as being roughly equally likely to be in class B. This is not true. x_a is surrounded by points in class B and thus we can be confident that it does indeed belong to that class. Meanwhile, x_b is far away from all points and thus we cannot be confident that it belongs to either class.

The posterior distribution is wide here because many possible θ values can separate the two classes and there is no single best model. In this classification problem, most models assign higher probability to x_a being in class B than x_b . The Bayesian approach aggregates these predictions and thus reflects greater confidence in x_a 's classification, even though individual models may disagree. This uncertainty quantification is a key advantage of the Bayesian approach.

4.2 Bayesian Occam's Razor

In this section I show how different models can be neatly compared under the Bayesian framework.

Bayesian models are also easier to compare to one another. Consider the principle of Occam's razor: a problem-solving technique stating that when presented with two competing hypotheses, one should always choose the simpler of the two. This principle can be applied to machine learning: complex models often outperform simpler alternatives because with more parameters they are better able to match the training data [Murphy, 2022]. This is not necessarily a good thing; fitting the training data too well often means the model is overfit and will perform poorly on unseen data. Therefore, when picking models, we should prefer simpler models assuming equivalent performance. But how much performance is worth sacrificing in the name of simplicity? The Bayesian approach provides an elegant solution to this problem. When comparing models, we can consider $p(D | m)$ where D represents observed data and m is the model being considered. The Bayesian approach seeks to understand the distribution of θ rather than the single best value. Therefore, we measure the expected value across all θ s for a model m :

$$\begin{aligned} p(D | m) &= \mathbb{E}_{p(\theta|m)} p(D | \theta, m) \\ &= \int p(\theta | m) p(D | \theta, m) d\theta \end{aligned}$$

Note that $p(\theta | m)$ is just the prior distribution for each model. Thus,

$$\int_{-\infty}^{\infty} p(\theta | m) d\theta = 1$$

A complicated model with a large parameter space must spread its probability mass thinly to satisfy this constraint. This comparatively increases $p(D | m)$ for simpler models, and decreases $p(D | m)$ for complicated models. This acts as a natural penalty for overly complicated models.

4.3 Mini Batch Algorithms

Most supervised machine learning algorithms, whether frequentist or Bayesian, rely on calculating the gradient of the log-likelihood. Traditionally, this calculation must be run across all data since the likelihood of all data is just the sum of the probability of each data point according to the model. This scales poorly in deep learning where the model contains many weights and is being trained with many data points. For example, with a million data points and a million weights, running inference for a single data point requires a trillion calculations. Since training requires running inference many times, this is often impractical.

To overcome this issue, it is commonplace to represent the whole dataset with a subset of the data. The likelihood of this subset is then scaled up to represent that of the full dataset. This adds noise to the likelihood and thus also to the gradient of the likelihood. Therefore, Bayesian models effective in deep learning must be trainable even when this “mini-batch” noise is present.

5 Introduction to MCMC

In this section, I provide an introduction to MCMC and discuss the most basic MCMC algorithms.

As previously mentioned, calculating the posterior distribution $p(\theta | D)$ for all possible values of θ is not always possible because θ is continuously distributed. To make matters worse, the evidence term

$$p(D) = \int p(\theta)p(D | \theta) d\theta$$

lacks a closed form solution.

Thankfully, MCMC can solve these issues and can also be easily adapted to sample from other distributions lacking closed form solutions. MCMC algorithms sample from the unnormalised posterior and estimate its distribution from the distribution of our samples. This is distinct from optimisation algorithms that find the mode of the posterior or likelihood. This is also distinct from variational inference methods which seek to stretch a known distribution to match the posterior. MCMC algorithms do a good job sampling from the area near θ_{MAP} in our posterior distribution. I refer to this area as the “area of interest”. Consider the logistic regression example from above where we estimated

$$p(y_i = A | D, \mathbf{x}_i) = \int p(y_i = A | \theta, \mathbf{x}_i) \cdot p(\theta | D) d\theta$$

using an MCMC algorithm to generate samples for $p(\theta | D)$. The prediction for $p(y_i = A | D, \mathbf{x}_i)$ will be most accurate if we sample values of θ near θ_{MAP} , since the predictions of these samples will have the greatest weight when we take the integral. Collecting many samples for θ far from θ_{MAP} is therefore not very useful.

This is where the true benefit of MCMC algorithms lies. Performing a grid search of θ is very difficult if θ is continuously distributed and of many dimensions. Even if you could, many samples would be wasted on θ s far from the area of interest.

5.1 Metropolis-Hastings Algorithm

One of the simplest MCMC techniques to approximate the posterior is the Metropolis-Hastings algorithm (often shortened to just Metropolis). We will first explain the algorithm and then provide an example. Proof that it satisfies detailed balance can be found in the appendix.

5.1.1 Explanation

The basic idea is to sample from a distribution on $\mathbb{R}^D$ where D is the number of dimensions in θ . Each sample x can thus be thought of as a θ where $\pi(x) = p(\theta | D)$. We sample from a function $q(x | x')$ where our next sample x depends only on the previous sample x' [Deligiannidis, b]. The algorithm generates a Markov chain X of these samples where X_t represents the value of the chain X at index t . After proposing a sample, the algorithm calculates $\alpha(X | X_{t-1})$ which represents the probability of accepting the sample and is defined as follows:

$$\alpha(X | X^{t-1}) = \min \left(1, \frac{\pi(X) q(X^{t-1} | X)}{\pi(X^{t-1}) q(X | X^{t-1})} \right)$$

The algorithm then accepts the sample with probability $\alpha(X | X^{t-1})$. This step of the algorithm is known as the ‘‘Hastings’’ correction step. If the Hastings correction is not accepted, it sets $X^t = X^{t-1}$. This means the proposed sample was rejected.

Following this strategy of proposing and accepting samples, a Markov chain is created such that

$$\forall x \in \mathbb{X}, \lim_{t \rightarrow \infty} P(X_t = x) = \pi(x).$$

The benefit here is that we never had to calculate $p(D)$ and were still able to sample from the posterior distribution. This works because Metropolis only relies on the ratio between $\pi(X^{t-1})$ and $\pi(X^t)$, not the values themselves.

The initial sample is unlikely to be close to θ_{MAP} . This means that the first few hundred samples are often just ‘‘walking’’ the Markov chain to the area of interest where the posterior is greatest. These samples are needed to explore the posterior; however, they dilute our final posterior estimate. Therefore, we often disregard our first few hundred samples as ‘‘burn in’’.

5.1.2 Intuition

To see how this algorithm makes sense, let us temporarily assume the q terms are equivalent. Doing so will make it clear why accepting worse samples is sometimes necessary. Note that the q terms often are equivalent regardless, as many proposal distributions are symmetric. For example, consider the case where the proposal distribution is a Gaussian centred at the previous sample. In this case,

$$\mathcal{N}(X | X^{t-1}, \sigma^2 \mathbf{I}) = \mathcal{N}(X^{t-1} | X, \sigma^2 \mathbf{I}).$$

If the q terms are equivalent, α simplifies down to

$$\alpha(X | X^{t-1}) = \frac{\pi(X)}{\pi(X^{t-1})}$$

Here, if $\pi(X) > \pi(X^{t-1})$ then we always accept. If this is the only way to accept a sample, we have turned our algorithm into an optimisation problem and as $t \rightarrow \infty$, X^t will converge to θ_{MAP} . Thus, we sometimes accept worse samples to properly mimic the distribution. For example, if $\pi(X) = (\frac{1}{2}) \cdot \pi(X^{t-1})$, then we accept the sample exactly half the time since

$$\alpha(X | X^{t-1}) = \frac{(\frac{1}{2}) \cdot \pi(X^{t-1})}{\pi(X^{t-1})} = \frac{1}{2}.$$

In order to understand the intuition behind the q terms, recognise that our Markov chain must reach detailed balance

$$\forall x, y \in \mathbb{X}, \pi(x) P(y | x) = \pi(y) P(x | y)$$

Here,

$$P(y | x) = q(y | x) \alpha(y | x)$$

so

$$\forall x, y \in \mathbb{X}, \pi(x) q(x | y) \alpha(x | y) = \pi(y) q(y | x) \alpha(y | x).$$

This means that for all states x, y the acceptance ratio is

$$\alpha(x | y) = \frac{\pi(y) q(y | x) \alpha(y | x)}{\pi(x) q(x | y)}.$$

The issue here is that $\alpha(x | y)$ depends on $\alpha(y | x)$, meaning that if one of the α values is less than one, the other will be greater than one. If we cap α to be no greater than one, we solve this problem. For example, if $\alpha(x | y) = 0.5$ then $\alpha(y | x)$ will be capped to one, meaning we can drop it from our equation. Hence,

$$\alpha(x | y) = \min \left(1, \frac{\pi(x) q(y | x)}{\pi(y) q(x | y)} \right).$$

5.1.3 Example

Consider the following example of using the Metropolis algorithm for linear regression:

$$\mathbf{x} = \{1, \dots, i\} \quad i = 10$$

$$y_i = mx_i + b + \epsilon_i$$

$$\epsilon_i \sim \mathcal{N}(0, 1)$$

We will define $\theta = \{m, b\}$ meaning for our problem, the parameter space for θ is $\mathbb{R}^2$. Let the ground truth value of m and b both be 1. We will use Metropolis to estimate the posterior distribution of θ , so

$$\pi(\theta) = p(\theta | X, Y).$$

Remember that we need to calculate $\pi(\theta)$ for a proposed θ when calculating α . Recall Bayes' rule which states that

$$p(\theta | X, Y) \propto p(X, Y | \theta) \cdot p(\theta).$$

For the prior term, let us use

$$p(\theta) = \mathcal{N}(0, I).$$

where I is the identity matrix. This means that $p(m)$ and $p(b)$ are independent.

For the likelihood term, assume each observation was independently generated by a Normal distribution centred at $mx_i + b$ with error $\sigma^2 = 1$:

$$p(X, Y | \theta) = \prod_{i=1}^{10} \mathcal{N}(y_i; mx_i + b, 1)$$

Finally, we can define the proposal distribution where

$$\theta' = \mathcal{N}(\theta, 0.1).$$

We denote the proposed state as θ' and the current state as θ . A proposal distribution like this is often referred to as a “Random Walk” proposal since our proposals are randomly generated. Since our proposal distribution is symmetric, α simplifies to

$$\alpha(\theta' | \theta) = \min \left(1, \frac{\pi(\theta')}{\pi(\theta)} \right).$$

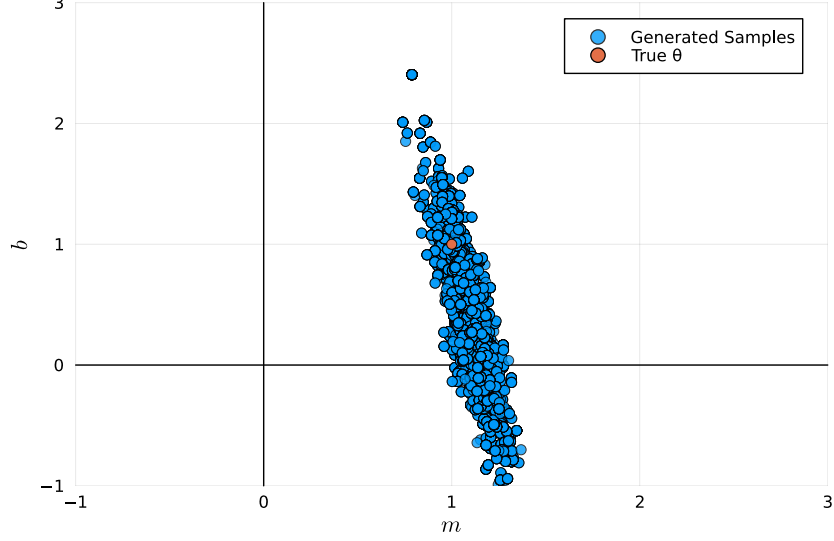


Figure 7: b and m values sampled with Random Walk Metropolis. θ' is sampled from $\mathcal{N}(\theta, 0.5)$ here.

We can change our proposal distribution to $\theta' \sim \mathcal{N}(\theta, 0.01)$. Proposed samples are now closer together such that our algorithm requires a longer burn-in time. However, our samples will also linger closer to θ_{MAP} , meaning we can better approximate $\pi(\theta)$ near this point.

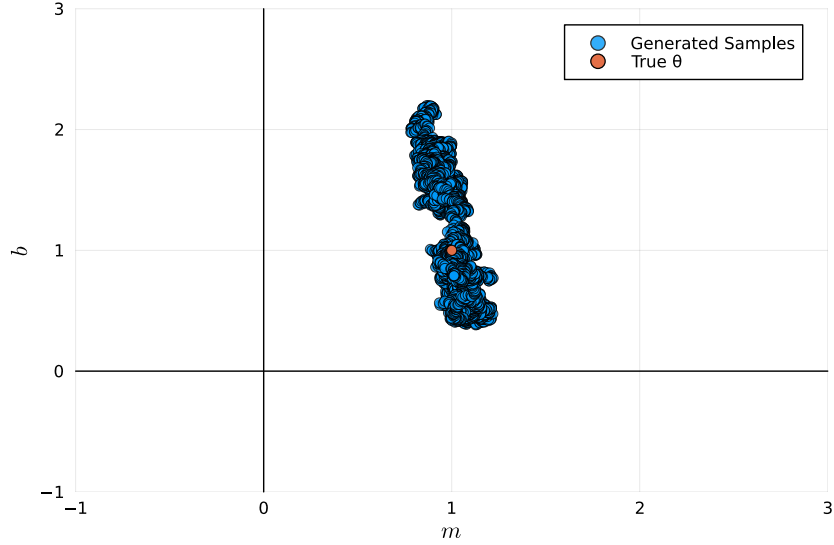


Figure 8: b and m values sampled with Random Walk Metropolis. θ' is sampled from $\mathcal{N}(\theta, 0.01)$ here.

We can also see that for both proposal distributions, there is greater variance in b samples than m samples. This means that the algorithm was more likely to accept samples where b was far from θ_{MAP} than m . This makes sense in the context of our problem since if $m \neq 1$, the likelihood of each data point will decrease dramatically as i approaches 10. Thus, if the newly proposed sample improves m , it will almost always be accepted. If it worsens m even a small amount, it will be rejected. This means that b will not be able to train since m dominates. This can be seen when examining the distributions of m and b :

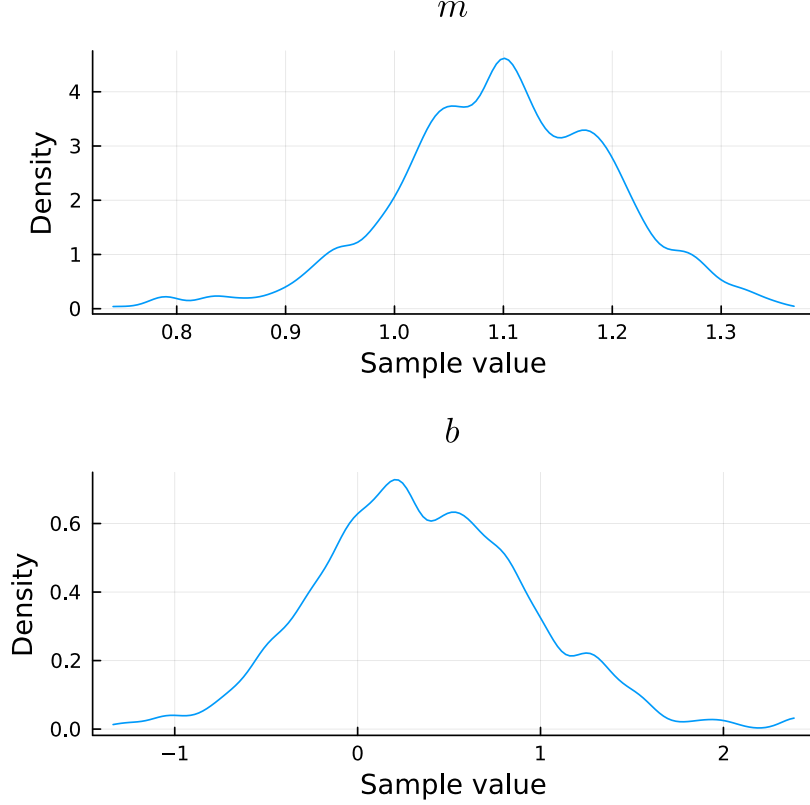


Figure 9: Density distributions of b and m sampled with the Metropolis algorithm using $\theta' \sim \mathcal{N}(\theta, 0.5)$

While our density distribution for m is reasonable given the true value of m , our distribution for b is sporadic. There exist strategies to make m less sensitive and therefore allow both parameters to converge. However, in a high-dimensional parameter space, it is possible for most parameters to move away from θ_{MAP} if only a few move towards it, such that the acceptance criterion is high enough for the algorithm to accept the new sample. This means that the Metropolis algorithm takes many samples to converge.

5.2 Langevin MCMC

Langevin MCMC [Chandra and Simmons, 2024] is an adapted version of Random Walk Metropolis where new samples are biased towards the area of interest.

When we use MCMC to sample θ we are often most interested in θ around θ_{MAP} . Since we initialise x randomly in our Markov chain, we want to sample near θ_{MAP} as quickly as possible. To achieve this, we can change our proposal distribution to favour samples nearer θ_{MAP} . Instead of

$$x' \sim \mathcal{N}(x, \sigma^2)$$

we can use

$$x' \sim \mathcal{N}(x + g(x), \sigma^2)$$

where

$$g(x) = \epsilon \nabla \pi(x)$$

Note that these proposals are not symmetric. This can be seen by comparing the forward and reverse proposal probabilities:

$$\begin{aligned} q(x | x') &= \mathcal{N}(x; x' + g(x'), \sigma^2) \\ q(x' | x) &= \mathcal{N}(x'; x + g(x), \sigma^2) \end{aligned}$$

This makes the process of calculating our acceptance ratio more complicated since we now need to include these proposal probability terms in our acceptance probability:

$$\alpha(x' | x) = \min \left(1, \frac{\pi(x') q(x | x')}{\pi(x) q(x' | x)} \right)$$

These proposal probabilities can cause our chain to get stuck: consider a scenario where our proposal deviation σ is small and ϵ is large. This is often the case, as we are often interested in $\pi(\theta)$ around θ_{MAP} and thus want to “walk” near there. To best do this, we want to quickly walk to θ_{MAP} (necessitating a large ϵ) and then make small steps near there (σ must be small).

This means that $g(x)$ is likely to be large in magnitude and x and x' likely differ mostly by a factor of $g(x)$ since σ is small. Therefore,

$$q(x' | x) = \mathcal{N}(x'; x + g(x), \sigma^2)$$

is a reasonably large number since $x' \approx x + g(x)$. However, the reverse probability

$$q(x | x') = \mathcal{N}(x; x' + g(x'), \sigma^2)$$

is likely small and near zero. This is the case because σ is small and x is very far from $x' + g(x')$.

This means that to have a chance of accepting, $\pi(x')$ must be much larger than $\pi(x)$ to compensate for $q(x | x')$ being so much smaller than $q(x' | x)$. However, this is not always the case. Recall $g(x) = \epsilon \nabla \pi(x)$. The gradient of the posterior is noisy and does not always point towards θ_{MAP} . This is how the chain gets “stuck”: if the gradient at the initial x does not accurately point towards θ_{MAP} , the chain will just repeatedly propose and reject similar samples. This notion of a chain getting stuck, where it is unable to explore the full posterior, is critical to designing effective MCMC algorithms.

5.3 Naive Stochastic MCMC

As previously discussed, mini-batch algorithms are preferable when dealing with large datasets. Both the Random Walk Metropolis and Langevin MCMC algorithms struggle in this regard.

Mini-batch algorithms approximate $\hat{\pi}(x)$, a mini-batch approximation for $\pi(x)$. Both Random Walk Metropolis and Langevin MCMC rely on an exact α calculation to decide whether to accept. However,

$$\min \left(1, \frac{\pi(x') q(x | x')}{\pi(x) q(x' | x)} \right) \neq \min \left(1, \frac{\hat{\pi}(x') \hat{q}(x | x')}{\hat{\pi}(x) \hat{q}(x' | x)} \right)$$

This means that any algorithm relying on an accurate Hastings correction step cannot be a mini-batch algorithm. We must use the full correction step:

$$\min \left(1, \frac{\pi(x') q(x | x')}{\pi(x) q(x' | x)} \right)$$

We can try to only use a Hastings step after several iterations to spread out the high cost of the correction. The issue then becomes that after several iterations, even more error is added to the system, meaning that the probability of accepting the correction decreases. Therefore, a lack of Hastings correction steps can lead to the chain getting stuck.

5.4 Stochastic Gradient Langevin Dynamics

Stochastic Gradient Langevin Dynamics (SGLD) [Welling and Teh, 2011] is an MCMC algorithm that also solves the “stuck chain” problem of Langevin MCMC. It works by combining two existing algorithms. The first, “Langevin Dynamics”, replicates the results of Metropolis-Hastings without needing a Hastings correction step. The second, “stochastic optimisation”, converts Langevin Dynamics to function as a mini-batch algorithm.

Langevin Dynamics can replicate the results of Metropolis-Hastings algorithms. Instead of using an acceptance step, the algorithm performs gradient descent and adds Gaussian noise to our new θ values to avoid converging to θ_{MAP} .

$$\Delta \theta_t = \frac{\epsilon}{2} \left(\nabla \log p(\theta_t) + \sum_{i=1}^N \nabla \log p(\mathbf{x}_i | \theta_t) \right) + \eta_t$$

$$\eta_t \sim \mathcal{N}(0, \epsilon)$$

This works by making use of the Fokker-Planck equation [Sato and Nakagawa, 2014] and relies on θ being continuous rather than a discrete set of samples. To simulate this, we use a small ϵ value to discretise the system.

To make this technique work with mini-batches, we can combine it with a class of methods known as stochastic optimisation [Robbins and Monro, 1951]. The core algorithm works as follows:

$$\Delta\theta_t = \frac{\epsilon_t}{2} \left(\nabla \log p(\theta_t) + \frac{N}{n} \sum_{i=1}^n \nabla \log p(\mathbf{x}_i | \theta_t) \right)$$

$$\epsilon_t = a(b + t)^{-\gamma}$$

Note that ϵ_t is never zero, ensuring convergence regardless of where the chain is initialised. Simultaneously, it decreases indefinitely, meaning the parameters will converge to the true posterior rather than bouncing around. Note that this is an optimisation algorithm; it is only when we add Gaussian noise that we create an MCMC algorithm that samples from the posterior. SGLD is that resulting MCMC algorithm. It is defined as

$$\Delta\theta_t = \frac{\epsilon_t}{2} \left(\nabla \log p(\theta_t) + \frac{N}{n} \sum_{i=1}^n \nabla \log p(x_t^i | \theta_t) \right) + \eta_t$$

where

$$\epsilon_t = a(b + t)^{-\gamma}, \quad \eta_t \sim \mathcal{N}(0, \epsilon_t).$$

The SGLD algorithm therefore contains two noise sources: injected Gaussian noise $\eta_t \sim \mathcal{N}(0, \epsilon_t)$ with variance $O(\epsilon_t)$, and mini-batch gradient noise with variance $O(\epsilon_t^2)$.

Thus, as $\epsilon_t \rightarrow 0$ the mini-batch noise disappears faster than the injected noise and the algorithm converges to Langevin Dynamics with stationary distribution $p(\theta | X)$.

SGLD is thus referred to as an algorithm of two phases. In the first phase, mini-batch gradient noise dominates as samples traverse towards the area of interest. In the second phase, injected noise dominates, and convergence to Langevin Dynamics lets us sample directly from the posterior. We thus discard samples as burn-in until we are in the second phase.

5.4.1 Example

Consider the following example where data is generated randomly from one of two Gaussians, with priors placed on the parameters θ_1, θ_2 :

$$p(\theta_1) = \mathcal{N}(0, \sigma_1^2) \quad p(\theta_2) = \mathcal{N}(0, \sigma_2^2)$$

$$x_i \sim \frac{1}{2} \mathcal{N}(\theta_1, \sigma_x^2) + \frac{1}{2} \mathcal{N}(\theta_1 + \theta_2, \sigma_x^2)$$

$$\sigma_1^2 = 5, \sigma_2^2 = 5, \sigma_x^2 = 2$$

We compute the posterior $p(\theta | x)$ using the above priors and mixture likelihood $p(x | \theta)$, plotted below:

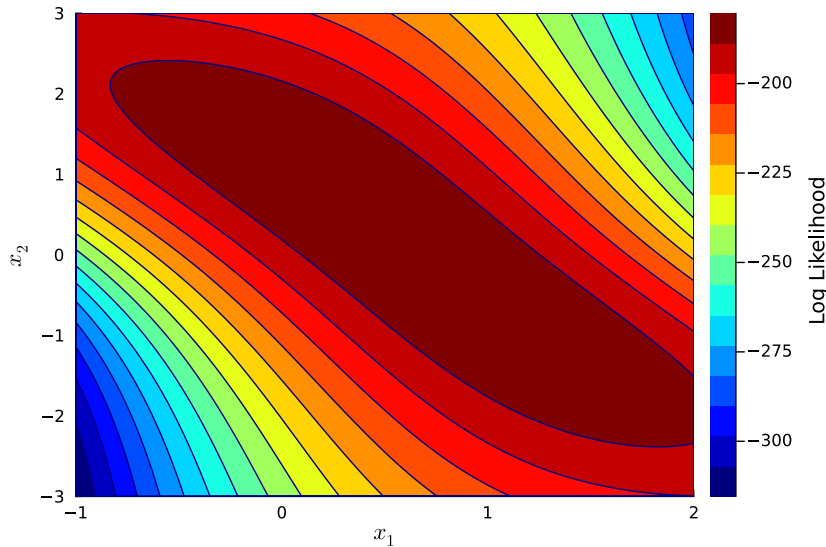


Figure 10: Contour plot showing the log posterior distribution of the mixed Gaussian example described above.

Using SGLD, we can sample from this posterior distribution. The results are shown below:

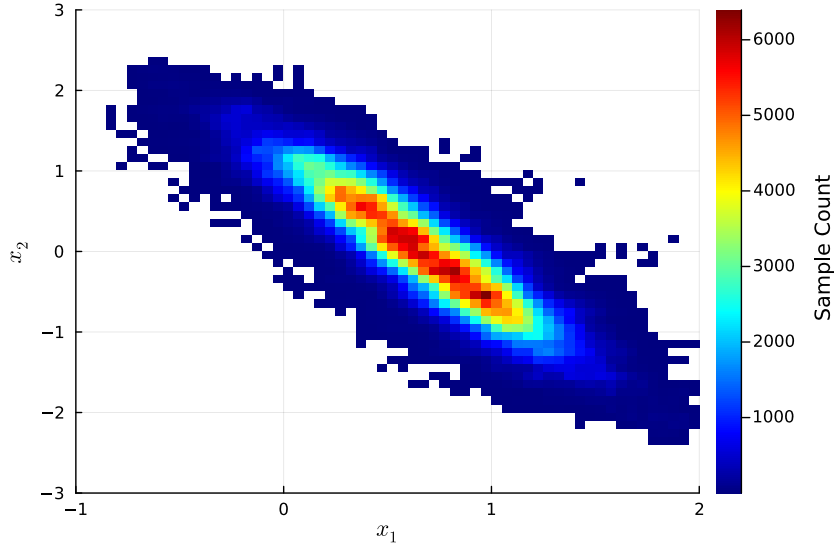


Figure 11: Histogram plot showing the samples of the log posterior collected by SGLD.

Finally, we can compare the variance of the two noise types.

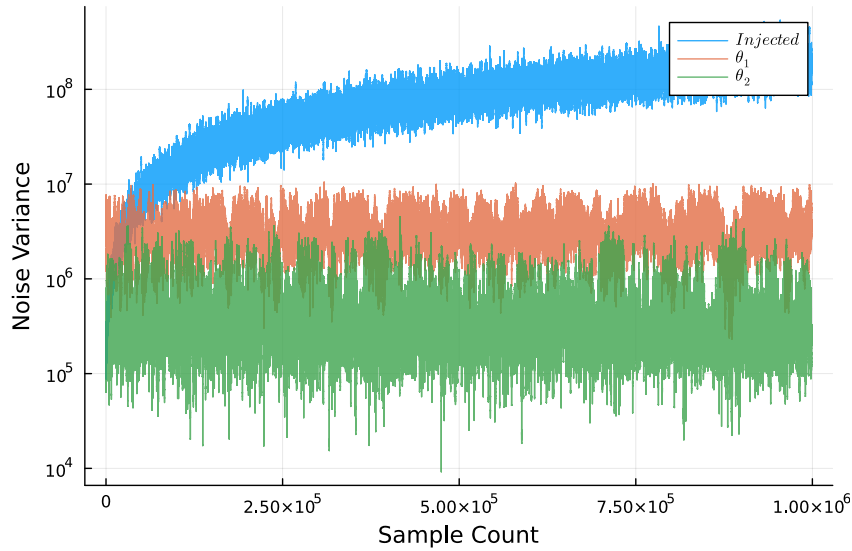


Figure 12: Plot comparing the variance of noise types. As time goes on, the injected noise dominates and the algorithm converges to Langevin Dynamics. The injected noise variance is measured here as the moving average of the squared noise added; it is essentially a function of ϵ_t . The noise from θ_1 and θ_2 is measured as the mean of the squared difference between the true gradient of the posterior and the mini-batch estimate.

Because the injected Gaussian noise $\eta_t \sim \mathcal{N}(0, \epsilon_t)$ has variance $O(\epsilon_t)$, and mini-batch gradient noise has variance $O(\epsilon_t^2)$, we expect injected noise to dominate as time goes on and ϵ_t goes to zero. This is what happens, as plotted above. This validates our idea of a two-phase algorithm. In the first phase, there is a mixture of mini-batch and injected noise. In the second phase, there is only injected noise.

6 Motivating Hamiltonian Dynamics

To accurately estimate the posterior distribution, we need to sample from $p(\theta \mid D)$ near θ_{MAP} . For example, if $p(\theta \mid D) = \mathcal{N}(0, 1)$, then taking samples where θ is very large will lead to poor predictions.

In this section, I show why if θ belongs to a high-dimensional space, then there is little benefit in sampling too close to θ_{MAP} . I show results illustrating how traditional MCMC algorithms perform poorly in high-dimensional spaces because of this additional constraint.

6.1 High Dimensional Spaces

In high-dimensional spaces, there is much more volume outside any given region around a point than within it. This can be understood intuitively by considering how the volume of a square or cube scales across dimensions.

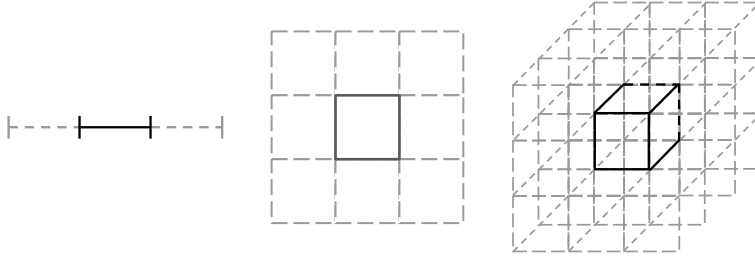


Figure 13: Consider an interval divided into three equal parts: the central segment makes up $\frac{1}{3}$ of the total length. In 2D, a square divided into a $3 \cdot 3$ grid has the central cell occupying $\frac{1}{9}$ of the area. In 3D, a cube divided into a $3 \cdot 3 \cdot 3$ grid has the central cell occupying $\frac{1}{27}$ of the volume.

For a D dimension cube, there will be $3^D - 1$ partitions touching the cube. This all means that when estimating the posterior $p(\theta \mid D)$, the value at the mode does not matter as it does not significantly contribute. Instead, what matters is the mass near but not exactly at the mode. We call this mass the typical set. The mode itself negligible volume in comparison to the typical set.

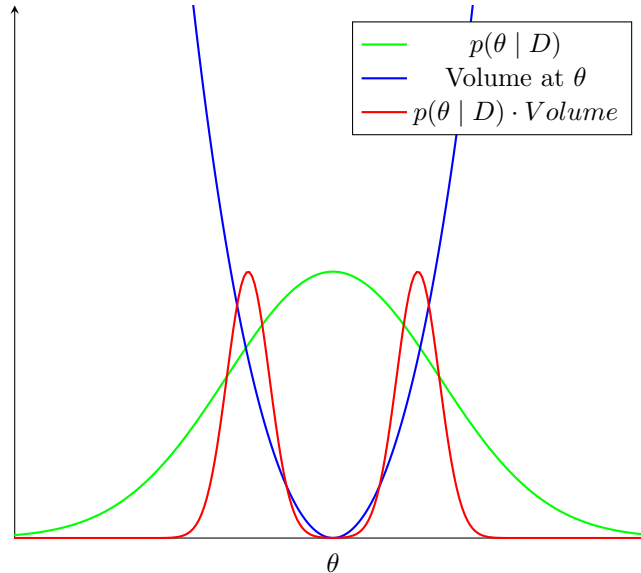


Figure 14: Visualisation of the typical set in a high-dimensional parameter space. θ_{MAP} lies at the mode of $p(\theta \mid D)$. At this point, and all other points in our high-dimensional space, the mass away from the point dominates the mass at the point. Thus, most of our posterior mass lies close but not too close to θ_{MAP} .

This means that if we assume our posterior distribution is a Gaussian, then the typical set will be a ring-like shape around θ_{MAP} . In high-dimensional spaces, therefore, being able to traverse this ring is very important. Too far from the centre of the ring and $p(\theta | D)$ will be close to zero. Too close to the centre of the ring and one ends up in a region with negligible volume.

6.2 MCMC Performance in High Dimensions

If the ring is a perfect circle, this is an easy task for Random Walk Metropolis. I can show this by creating an example function that is ring-shaped and collecting samples using Metropolis.

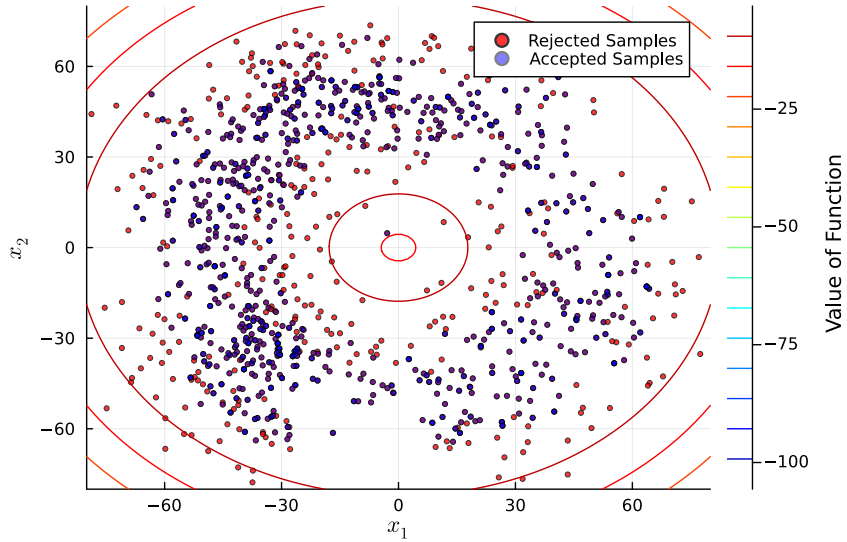


Figure 15: Random Walk Metropolis on a ring-shaped function. We can see that most rejected samples occur on either edge of the ring shape. This makes sense, as the function decays if the radial distance gets too small or large.

Random Walk Metropolis works well because proposals are almost always accepted as long as they do not leave the ring, which is unlikely due to the geometry of the shape.

If the posterior is not a simple Gaussian but instead a more complicated shape, the typical set will not be a circular ring. Assuming θ_{MAP} exists, the typical set will always form a closed loop. However, the ring shape could contain sharp turns or kinks. To simulate this case, we apply Random Walk Metropolis to a sinusoidal function. Each wave acts as a kink that the algorithm must traverse. One can imagine that the peaks of the graph represent θ_{MAP} , where volume is near zero, and the troughs represent θ values far from θ_{MAP} , where the posterior is near zero.

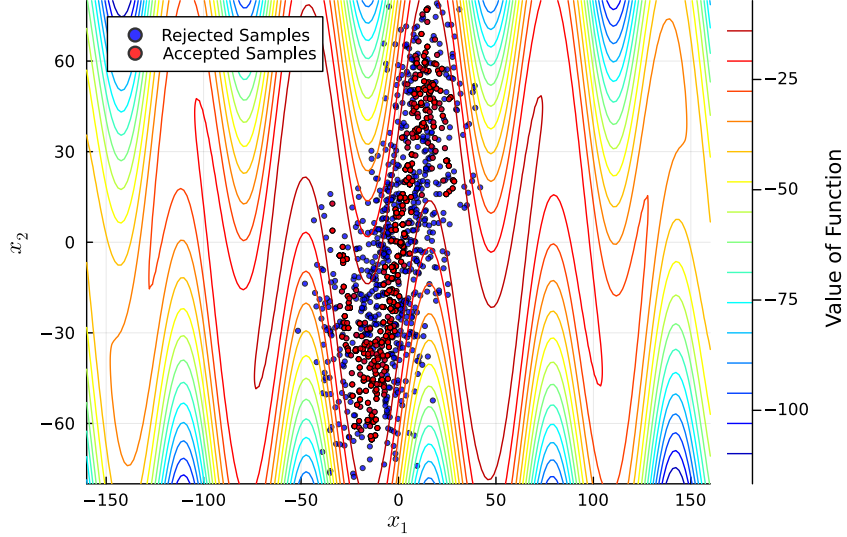


Figure 16: Random Walk on the sinusoidal posterior. Many proposals are rejected since the shape of the posterior contains many kinks. This results in a biased posterior estimate as the algorithm frequently gets stuck. The entire right side of the graph is unexplored.

If we increase the variance in our Gaussian proposal distribution, we can try to “jump” out of the region where we are stuck. However, this will only increase the percentage of rejected samples and will not help us explore beyond the kinks.

Alternatively, one could attempt to use a Langevin technique to move up and down the sine waves using the gradient. The issue is that the gradient does not naturally push samples left and right through the waves. The gradient is noisy and samples do not neatly “flow” through the waves. Hamiltonian dynamics, and Hamiltonian Monte Carlo, provide a solution to this issue.

7 Hamiltonian Dynamics

In this section, I first explain Hamiltonian Dynamics and its use in MCMC and optimisation algorithms. I then explain and show results confirming that such techniques can effectively explore the typical set.

7.1 Intuition

Imagine a ball on top of a hill. We can describe its state using a combination of its position q and momentum p [Betancourt, 2018]. The total energy of the ball can be represented using Hamiltonian Dynamics. We can say

$$H(q, p) = K(p) + U(q).$$

where K is the kinetic energy of the ball, U is the potential energy of the ball, and $H(q, p)$ is the sum of the two [Deligiannidis, a].

Hamiltonian systems evolve according to

$$\frac{dq}{dt} = \frac{\partial H}{\partial p}, \quad \frac{dp}{dt} = -\frac{\partial H}{\partial q}.$$

Intuitively, $\frac{dq}{dt} = \frac{\partial H}{\partial p}$ captures that a particle with more momentum moves faster, while $\frac{dp}{dt} = -\frac{\partial H}{\partial q}$ captures that a particle loses momentum as it moves into regions of higher potential energy. The total energy H is always conserved: as the ball moves, any change in kinetic energy corresponds to an equal and opposite change in potential energy.

We can exploit Hamiltonian dynamics in both optimisation and MCMC algorithms. In both cases, we treat our parameters θ as a position q in parameter space, and introduce p as an auxiliary variable to represent the momentum. We can write the joint distribution as

$$\pi(q, p) \propto \exp(-H(q, p)) = \exp(-U(q) - K(p))$$

where

$$U(q) = -\log p(q \mid D), \quad K(p) = \frac{1}{2} p \cdot p^T$$

Since the exponent separates into two independent terms, we can factorise

$$\pi(q, p) = \exp(-U(q)) \cdot \exp(-K(p)).$$

Since $K(p)$ is independent of q , we can say that

$$\pi(q) \propto \exp(-U(q)).$$

Therefore, samples of q collected from the joint $\pi(q, p)$ are automatically samples from the posterior, regardless of p . The momentum is auxiliary and can be discarded.

Since Hamiltonian dynamics is a continuous system, we must use a numerical integrator to simulate it in discrete steps. Two common choices are the Euler method and the leapfrog integrator. A discussion of both can be found in the appendix. Importantly, both introduce small discretisation errors that can be accounted for with a Hastings correction step. In practice, algorithms sometimes ignore these errors.

7.2 Hamiltonian Dynamics for Optimisation

Since the energy of the system is constant, samples “orbit” around θ_{MAP} rather than converging to it.

To correct this, we introduce friction: after each step we reduce the momentum by a constant factor. When combined with mini-batch gradients, this technique is known as Stochastic Gradient Descent with Momentum (SGDM).

Consider the following example where θ belongs to a 2D parameter space. Let us initialise θ at $(1, 1)$ and define the posterior to be a normal distribution with mean $(0, 0)$. Set the covariance matrix to be the identity matrix and initialise momentum randomly:

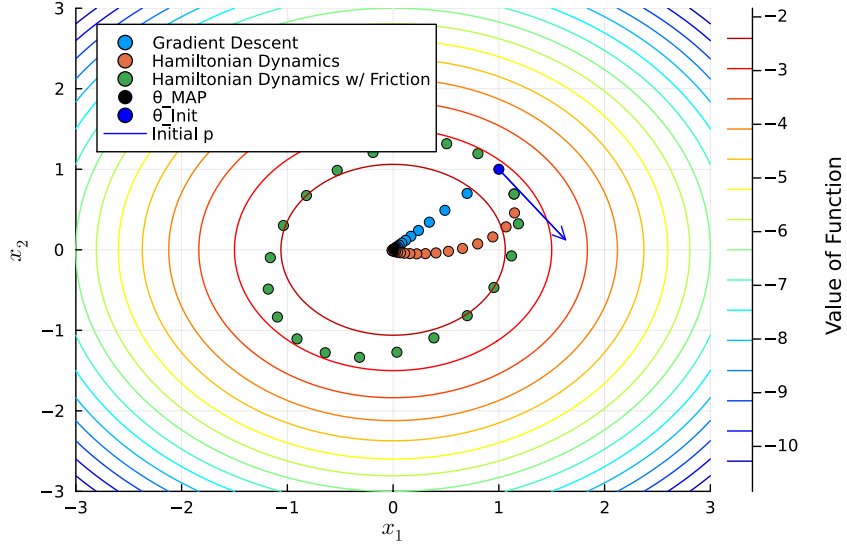


Figure 17: Example of Hamiltonian dynamics with and without friction. Without friction, samples orbit around the mode. With friction, samples descend towards the mode. In most cases, the gradient of the function at the mode is near zero. This means that gradient descent can get stuck because at each iteration, the gradient approaches zero and θ slows down. If the friction term is configured correctly, Hamiltonian dynamics can mitigate this problem by using momentum to travel to the mode much faster.

Frictionless Hamiltonian dynamics orbits around θ_{MAP} repeatedly. Momentum is greater when closer to θ_{MAP} . This can be visualised by samples being further apart when closer to θ_{MAP} . With friction, the samples travel to θ_{MAP} faster than or at the same rate as standard SGD. Momentum-based optimisation techniques such as Adam can be thought of as variants of SGDM and are popular in the machine learning community.

7.3 Hamiltonian Monte Carlo

To create an MCMC algorithm using Hamiltonian Dynamics, note that the orbit behaviour shown above contains little diversity of samples. To break this, we run the simulation for some fixed number of steps, take a sample, and then resample the momentum p before the next set of steps. This allows the chain to explore the full distribution.

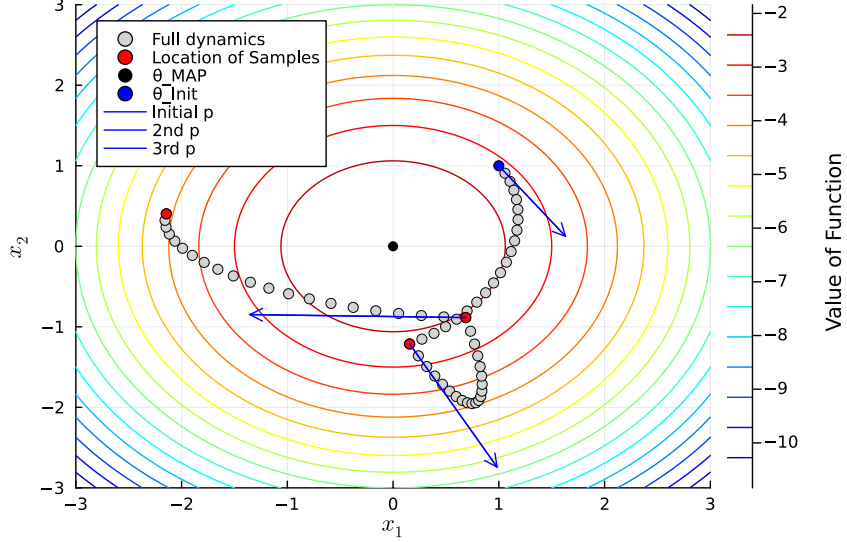


Figure 18: Example of the first few iterations of Hamiltonian Monte Carlo. After a set number of steps, a new momentum is sampled.

For Hamiltonian Monte Carlo to work we must preserve detailed balance. We can show this by showing that HMC is just a Metropolis-Hastings algorithm with a more complicated proposal step. The only complication is that we must augment the acceptance ratio to account for both q and p :

$$\alpha(q', p' | q, p) = \min \left(1, \frac{\pi(q', p') P(q, p | q', p')}{\pi(q, p) P(q', p' | q, p)} \right)$$

All samples of p and q with the same initial momentum sample will be time-reversible, meaning that

$$P(q', p' | q, p) = P(q, p | q', p').$$

This is the case because if we negate the momentum at any point in the trajectory from (q, p) to (q', p') , it exactly retraces its path back from (q', p') to (q, p) . Intuitively, think of a ball rolling down a half-pipe: once it reaches the other side at (q', p') , reversing the momentum sends it back along the same trajectory to (q, p) . Thus the acceptance ratio simplifies to

$$\alpha(q', p' | q, p) = \min \left(1, \frac{\pi(q', p')}{\pi(q, p)} \right)$$

Note that

$$\pi(q', p') \approx \pi(q, p)$$

since energy is preserved in a Hamiltonian system. The only difference between the two is discretisation error introduced by the numerical integrator. Therefore, if we use a Hastings correction step here, we can be confident that detailed balance is preserved for all samples with the same initial momentum.

When we resample p , we are changing the energy of the system. If we assume we are sampling momentum from a zero-centred Gaussian, we can intuitively understand why this satisfies detailed balance. If we sample a large, unlikely momentum p' that shoots us far from the area of interest, then $P(q', p' | q, p)$ will be very small even if $\pi(q, p)$ is large. Likewise, $\pi(q', p')$ will be very small, but the reverse transition probability $P(q, p | q', p')$ will be large since samples are pulled back towards the area of interest near θ_{MAP} .

Looking at the detailed balance equation

$$\pi(q, p) P(q', p' | q, p) = \pi(q', p') P(q, p | q', p'),$$

we thus have one small and one large term on each side. This intuitively justifies why resampling p satisfies detailed balance.

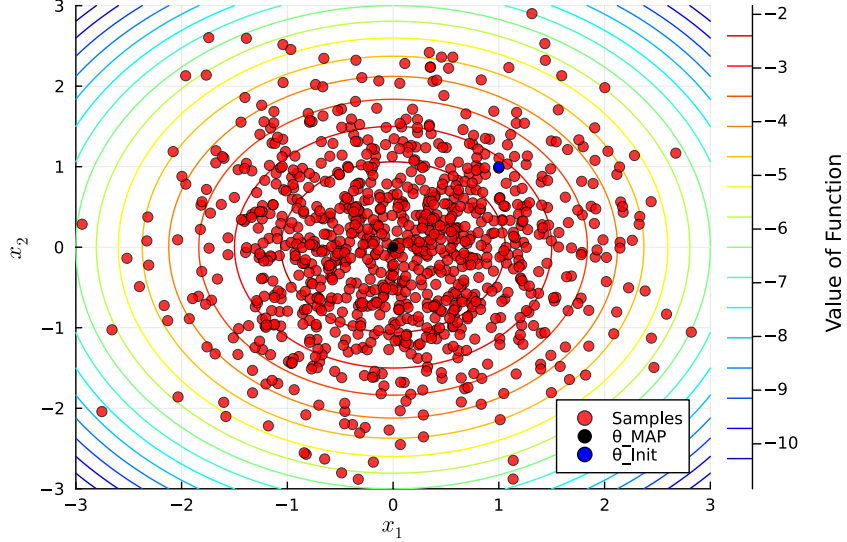


Figure 19: HMC simulation result. One can see that the normal distribution is recovered.

7.3.1 Exploring the Typical Set with HMC

Recall the issue of exploring the typical set. I have established that if the typical set is ring-shaped, then Random Walk Metropolis can explore it well. Below is the result of using HMC on the same problem.

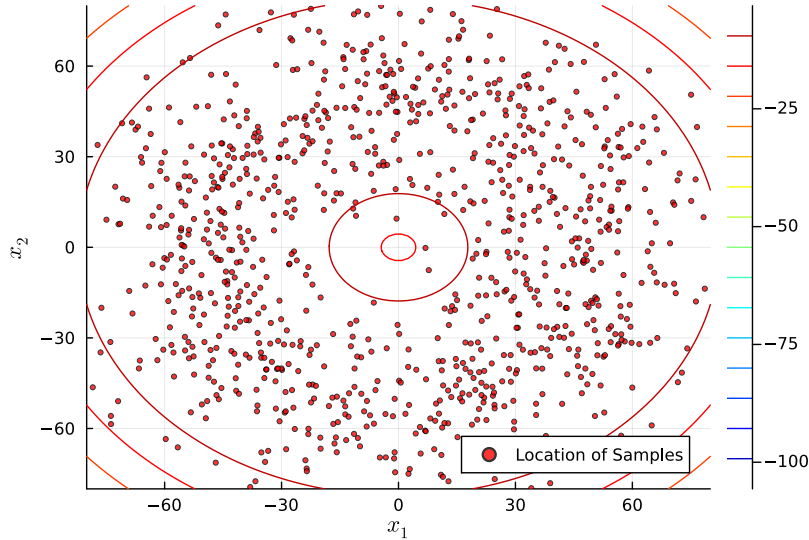


Figure 20: HMC samples on the ring example. HMC does a good job sampling from the typical set.

Previously, I showed that Random Walk Metropolis struggled on the sinusoidal example because the geometry of the posterior means that many proposed samples were rejected.

HMC does not suffer from the same fate. Even if momentum values are initially sampled randomly, the momentum changes according to the gradient of the posterior at each step. This lets HMC traverse through the waves of the posterior similarly to water running down a water slide.

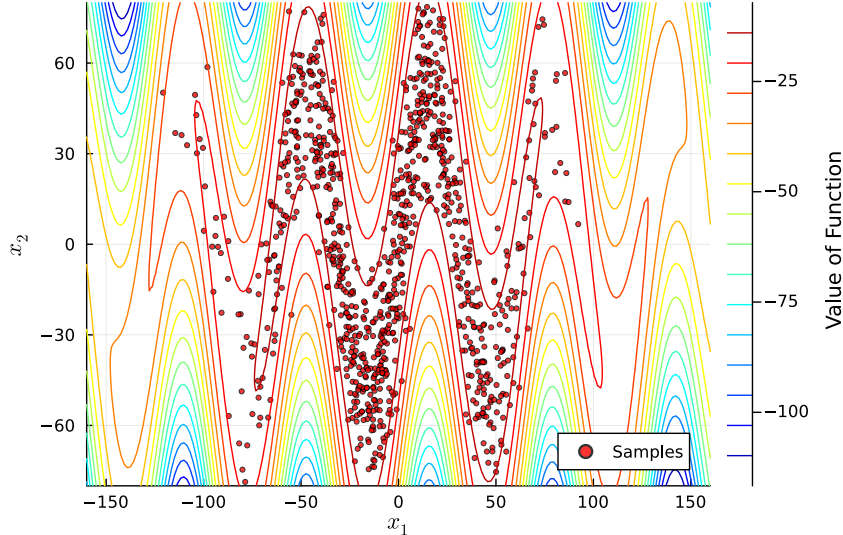


Figure 21: HMC samples on the sinusoidal example. HMC does a good job traversing beyond the kinks of the waves and explores all parts of the typical set.

For both posterior shapes, we can see that HMC is preferable or equivalent to Random Walk Metropolis. If nothing else, Metropolis rejects many samples, wasting computational resources. Even if Hamiltonian Monte Carlo gets stuck in a kink somewhere, each sample differs from the previous, meaning samples are at least exploring some region, even if not the full typical set.

7.4 Stochastic Gradient Hamiltonian Monte Carlo

I have now shown the benefit of Hamiltonian Monte Carlo. One problem still remains: Hamiltonian Monte Carlo is not naturally a mini-batch algorithm. I have previously discussed why mini-batch algorithms are preferable for deep learning. This motivates Stochastic Gradient Hamiltonian Monte Carlo (SGHMC).

In this section, I provide a brief overview of the algorithm before discussing results in the Results section.

SGHMC [Chen et al., 2014] works much like HMC. Since using mini-batches adds noise to the system, energy is added and detailed balance is broken. This presents a significant problem because the noise from the stochastic gradient can accumulate and radically change the trajectory of our Hamiltonian system.

To fix this, we introduce a drag coefficient similar to SGDM. The core idea is that the reduction in energy from the drag will cancel out the added energy from the stochastic gradient.

How much drag to add depends on how much noise is added from the mini-batch approximation of the full gradient. Since this is hard to estimate, we can rely on the injected noise trick used in SGLD. This works by defining ϵ_t such that as $t \rightarrow \infty$, $\epsilon_t \rightarrow 0$. We then inject noise whose variance scales with ϵ_t . Similar to SGLD, this injected noise will dominate the mini-batch noise after many samples. Since we know the variance of this injected noise, we know how much friction to add.

SGLD can be seen as a limited form of SGHMC. If the friction term is very high, then each sample will move only a small amount in the direction that minimises the gradient before the energy runs out. In this case, each SGHMC sample consists of injected noise, stochastic noise, and a small step in the direction that minimises loss. This is exactly what makes up SGLD.

8 Results

I have now explained the intuition behind several mini-batch MCMC and full-batch MCMC algorithms. I now present my comparison of the algorithms. I argue that when the data does not clearly point to a single θ as being the true θ , MCMC algorithms outperform their optimisation-based alternatives. I make these arguments by analysing performance on two datasets: the Moon dataset and MNIST [LeCun and Cortes].

8.1 Two Moon Dataset Analysis

Analysing the performance of mini-batch MCMC methods on the Moon dataset illustrates how such methods can better represent uncertainty in areas far away from existing data. To do this, I compare SGD and SGLD on the Moon dataset.

The Moon dataset is a two-class logistic regression problem in 2D. I generated the data and problem myself.

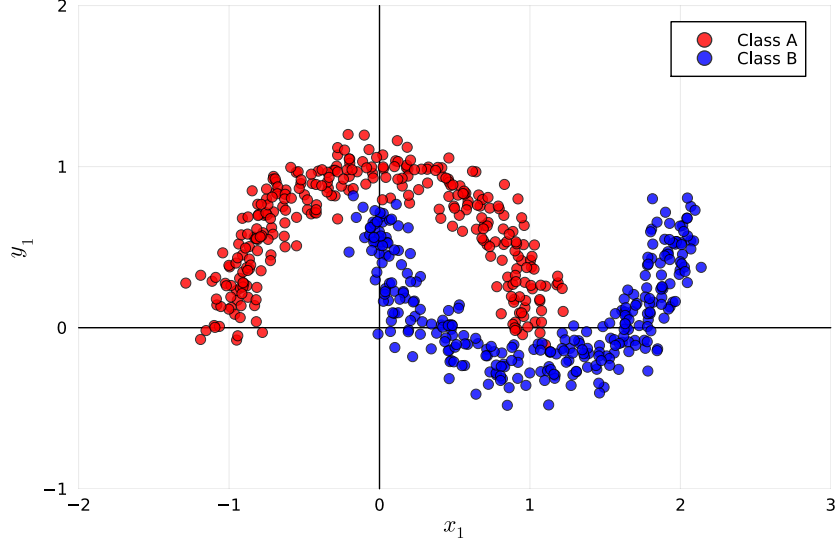


Figure 22: Moon Data

This data is not linearly separable as there is no line that can perfectly divide the data. We thus use a neural network to represent this non-linear relationship. Our network has two inputs, one for each dimension, five hidden nodes, and one output layer to represent the probability of the input belonging to class A. We can use autograd or similar technologies to find the gradient of all weights in the neural network with respect to the posterior.

With this gradient, we can perform gradient descent or ascent to find θ_{MLE} . Using θ_{MLE} , we can generate a contour of predicted classifications by feeding every possible coordinate into the neural network. In the plot below, we can see a steep S-curve between the crescents since the two classes are strictly separated yet very close to one another.

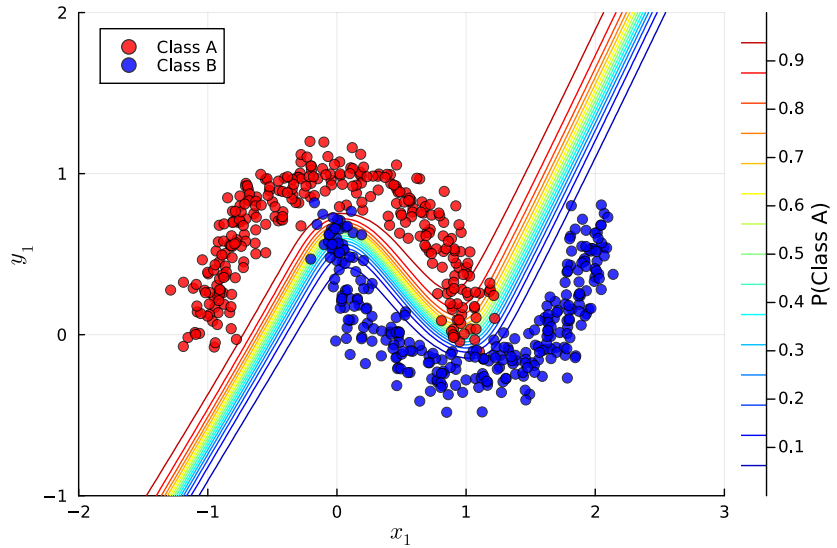


Figure 23: SGD Moon Data

We can also use an MCMC algorithm here. Below is the result of using SGLD. We make predictions using each θ sample and take the average of those predictions. The same tight S-curve is present between crescents. The difference is that there is greater uncertainty to the left and right of the crescents where the S-curve flattens.

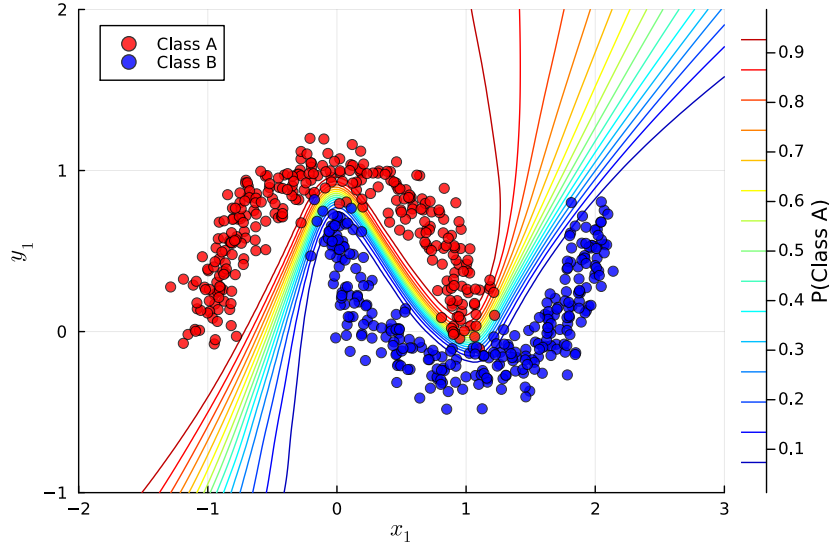


Figure 24: SGLD Moon Data

This example is similar to the logistic regression problem discussed previously in the motivation section. The biggest difference is that here our model is non-linear, as seen in the non-linear S-curve.

Intuitively, we should have less confidence labelling points far from our data. SGD struggles to capture this because it only uses a single set of weights. Using a single set of weights to make predictions can be thought of as using a single sample from an MCMC algorithm. Regardless of the quality of that sample, the weights are assumed to be true and thus cannot represent uncertainty. This is the case regardless of whether the weights represent a linear or non-linear model.

8.2 MNIST Dataset Analysis

MNIST is a popular dataset of handwritten digits. I implemented SGD, SGDM, SGLD and SGHMC and trained them on the dataset. My results show how MCMC algorithms perform best when the data is noisy and the posterior is wide.

The MNIST dataset contains 60,000 examples for training and another 10,000 for testing. The experimental setup follows that of [Chen et al., 2014] for both the optimisation and sampling methods.

The results below show that the MCMC methods slightly outperform the frequentist ones, and that the Hamiltonian approaches slightly outperform the non-Hamiltonian approaches.

Note that the y -axis here represents test error. For the optimisation algorithms (SGD, SGDM), this is defined as

$$\frac{1}{n} \sum_{i=1}^n \mathbf{1}(\hat{y}_i \neq y_i)$$

where n is the number of data points in the test set. Recall that $\hat{y}_i$ is calculated from the neural network and depends on θ_{MLE} . We can write this prediction as

$$\hat{y}_i = f(\theta)$$

where our neural network is represented as a function that takes its weights as an argument.

For the Bayesian algorithms, there is no singular θ from which to find the test error. Instead, we make predictions using all sampled θ values and only then compare the test error of our prediction $\hat{y}$. Thus, our test error is

$$\frac{1}{n} \sum_{i=1}^n \mathbf{1}(f(\theta) \neq y_i)$$

where

$$f(\theta) = \frac{1}{s} \sum_{j=0}^s f(\theta_j)$$

and $f(\theta_j)$ represents the predictions from the neural network made with the θ from the j th sample of the algorithm. If we knew the posterior distribution exactly, we could write

$$\hat{y}_i = \int_{-\infty}^{\infty} f(\theta) \cdot p(\theta | D) d\theta$$

Since we cannot find the posterior, we instead rely on the generated samples being accurate.

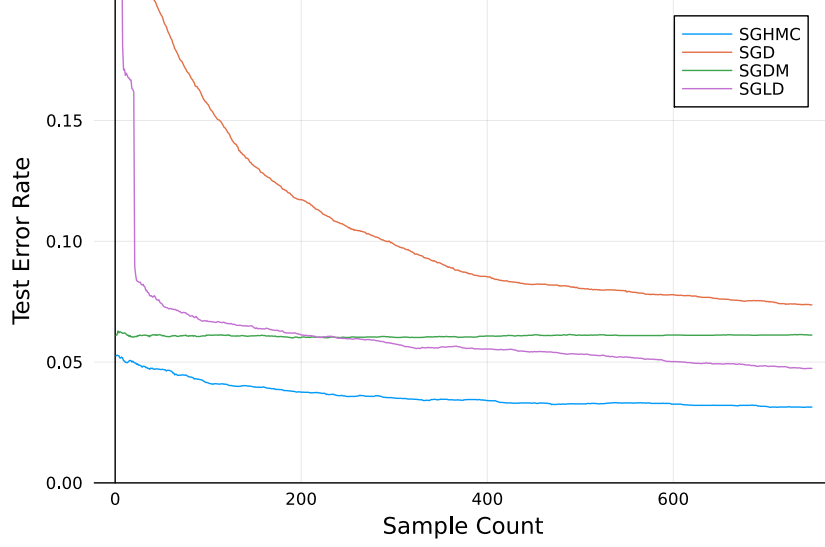


Figure 25: MNIST Full Data

I have previously shown how Hamiltonian dynamics can outperform both optimisation and non-Hamiltonian MCMC algorithms, as they are better able to traverse complicated posteriors without getting stuck. This may explain why SGDM and SGHMC perform well.

However, all four algorithms achieve a similar test error. This is because the posterior is very concentrated around a single set of weights, meaning that most samples will be near θ_{MAP} . This is similar to the tight S-curve between the moons in the previous example. In both examples, the data points to a singular θ being best. The argument for the Bayesian approach is weakest here, since the MCMC algorithms are computationally expensive and do not lead to significantly better results.

If we reduce our training set to a twelfth of its original size, the posterior distribution flattens as we have less certainty in the ground truth value of θ . In this case, any single θ found by an optimisation method will not be particularly good. MCMC algorithms perform better because they represent the full width of the posterior and spread θ samples across all areas of interest. This is when the Bayesian approach is at its best, since a better representation of the posterior leads to better predictions.

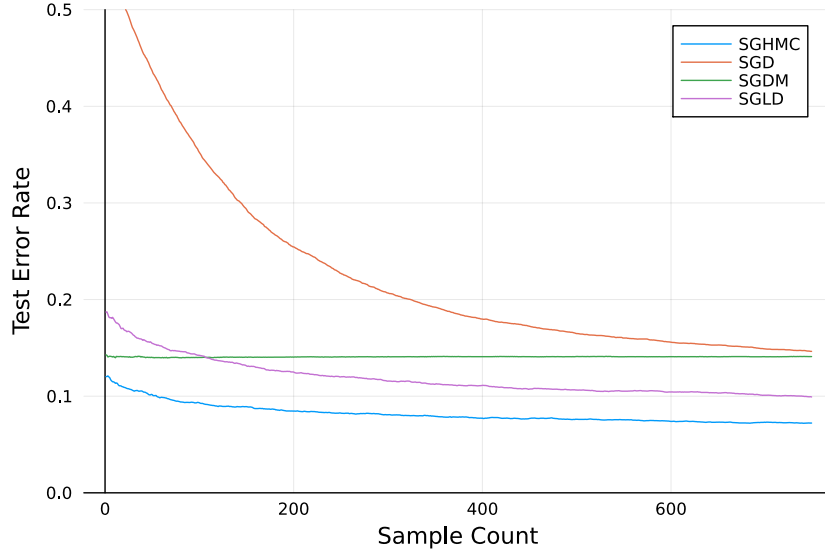


Figure 26: MNIST 5000 Data

All four algorithms support mini-batch training and performed competitively in this setting. This is significant for reducing computational cost, and will grow more relevant as dataset sizes continue to increase.

9 Future Work

In this section, I discuss potential areas of future research and how this dissertation could be expanded with more time available.

More advanced Hamiltonian Monte Carlo algorithms could be adapted to support mini-batch usage. In 2015, two of the three co-authors of the SGHMC paper showed that any MCMC algorithm could be converted to a stochastic equivalent [Ma et al., 2015]. This opens the door to more advanced HMC algorithms being adapted to support mini-batch learning.

For example, the No-U-Turn Sampler [Hoffman and Gelman, 2011] (NUTS) improves upon sampling issues in HMC. Traditional HMC simulates the dynamics for a fixed length of time before taking a sample. If unlucky, this can result in taking a sample at almost the exact point where the simulation started. NUTS keeps track of where samples are taken and only stops the simulation to collect samples when they are far away from the current sample. This has been shown to lead to improved performance compared to standard HMC and could potentially support mini-batching assuming the results from Chen et al. 2014 hold. Further research could explore this potential.

Another area to explore is the efficacy of existing mini-batch MCMC algorithms versus their full-batch alternatives. Clearly, HMC will perform better than SGHMC over the same number of samples. The question to address is how much performance is worth sacrificing to use a mini-batch algorithm. Presumably, SGHMC performs better with larger batch sizes, which are more computationally costly. Quantitatively exploring these trade-offs would help users understand when to use which algorithm.

A Comparison of Iterators

To simulate Hamiltonian dynamics, various iterators can be used.

The simplest iterator, known as the Euler iterator, is used in the original paper presenting SGHMC and the implementation presented in this dissertation. It is defined as follows:

$$p_i(t + \epsilon) = p_i(t) - \epsilon \nabla_{q_i} U(q(t))$$

$$q_i(t + \epsilon) = q_i(t) + \epsilon \nabla_{p_i} K(p(t))$$

This implementation can be improved upon by using the leapfrog iterator.

$$p_i(t + \epsilon/2) = p_i(t) - \frac{\epsilon}{2} \nabla_{q_i} U(q(t))$$

$$q_i(t + \epsilon) = q_i(t) + \epsilon \nabla_{p_i} K(p(t + \epsilon/2))$$

$$p_i(t + \epsilon) = p_i(t + \epsilon/2) - \frac{\epsilon}{2} \nabla_{q_i} U(q(t + \epsilon))$$

The leapfrog iterator requires two gradient calculations per step, which is expensive, but outperforms the simpler Euler iterator as seen below:

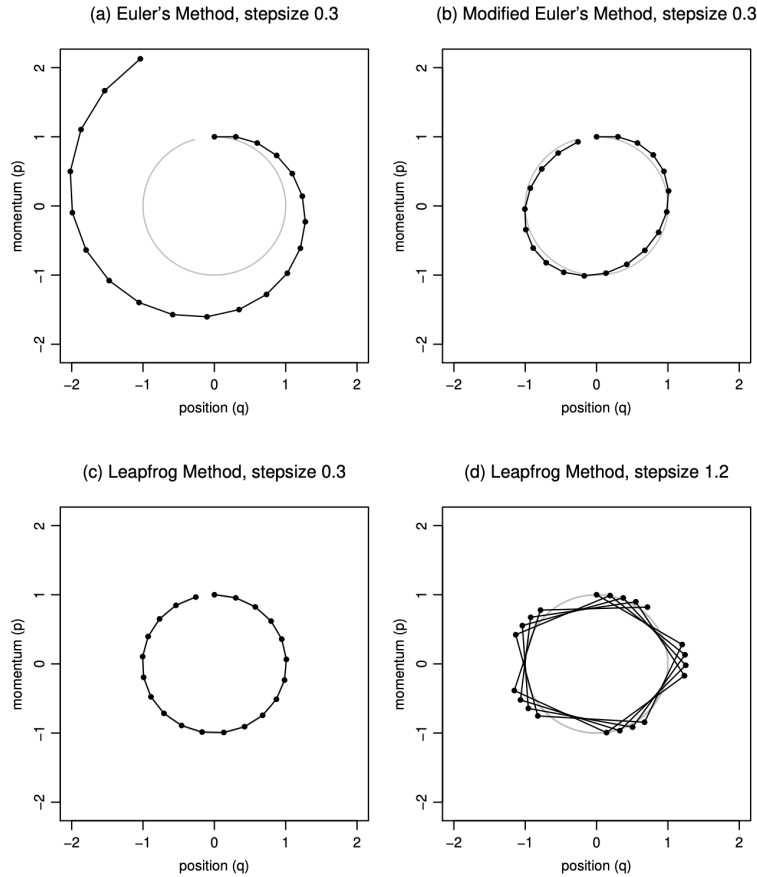


Figure 27: Leapfrog and Euler method comparison. Graph from [Neal, 2012].

The iterators can explore more of the posterior if the step size is large. The leapfrog iterator being more accurate even with a larger step size is thus a significant advantage. Too much inaccuracy requires a costly Hastings correction step which all algorithms seek to avoid.

B Proof of the Transition Kernel

The transition kernel of the Markov chain generated by the Metropolis-Hastings algorithm is defined as follows:

$$K(x^{(t-1)}, x^{(t)}) = \alpha(x^{(t)} | x^{(t-1)}) q(x^{(t)} | x^{(t-1)}) + (1 - \alpha(x^{(t-1)})) \delta_{x^{(t-1)}}(x^{(t)})$$

Proof. For any $A \subset \mathbb{X}$,

$$\begin{aligned} & P(X^{(t)} \in A \mid X^{(t-1)} = x^{(t-1)}) \\ &= P(X^{(t)} \in A, \text{proposal accepted} \mid X^{(t-1)} = x^{(t-1)}) \\ &+ P(X^{(t)} \in A, \text{proposal rejected} \mid X^{(t-1)} = x^{(t-1)}) \end{aligned}$$

This means that $X^{(t)} \in A$ can happen in two cases: one where the last proposal was accepted and one where it was rejected. The rejected case expands with Bayes' rule.

$$\begin{aligned} & P(X^{(t)} \in A, \text{proposal rejected} \mid X^{(t-1)} = x^{(t-1)}) \\ &= P(X^{(t)} \in A \mid \text{proposal rejected}, X^{(t-1)} = x^{(t-1)}) \\ &\cdot P(\text{proposal rejected} \mid X^{(t-1)} = x^{(t-1)}) \end{aligned}$$

At this point, we can start assigning values to these probabilities.

$$P(\text{proposal rejected} \mid X^{(t-1)} = x^{(t-1)}) = 1 - a(x^{(t-1)})$$

This is one minus the probability of accepting a proposal. It is also accurate to say

$$P(X^{(t)} \in A \mid \text{proposal rejected}, X^{(t-1)} = x^{(t-1)}) = \int_A \delta_{x^{(t-1)}}(x^{(t)}) dx^{(t)}.$$

Here, $X^{(t)} = X^{(t-1)}$ since the last proposal was rejected. The probability of $X^{(t)} \in A$ is thus the sum of all possible ways that $X^{(t)} \in A$ can happen. This is the sum rule, expressed as an integral since A is not discrete. For $X^{(t)} \in A$ to be true, $X^{(t)}$ must equal $X^{(t-1)}$, which is what the Dirac mass achieves.

Finally, we can say

$$\begin{aligned} & P(X^{(t)} \in A, \text{proposal accepted} \mid X^{(t-1)} = x^{(t-1)}) \\ &= \int_A \alpha(x^{(t)} \mid x^{(t-1)}) q(x^{(t)} \mid x^{(t-1)}) dx^{(t)}. \end{aligned}$$

We integrate over A using the same sum rule: for the proposal to be accepted and land in A , the proposed point $x^{(t)}$ must lie in A and be accepted with probability $\alpha(x^{(t)} \mid x^{(t-1)})$ under the proposal density q .

Finally, recall that

$$P(X^{(t)} \in A \mid X^{(t-1)} = x^{(t-1)}) = \int_A K(x^{(t-1)}, x^{(t)}) dx^{(t)}.$$

Combining terms:

$$\begin{aligned} \int_A K(x^{(t-1)}, x^{(t)}) dx^{(t)} &= \int_A \alpha(x^{(t)} \mid x^{(t-1)}) q(x^{(t)} \mid x^{(t-1)}) dx^{(t)} \\ &+ \int_A \delta_{x^{(t-1)}}(x^{(t)}) dx^{(t)} \cdot (1 - a(x^{(t-1)})) \end{aligned}$$

Since this equality holds for all measurable $A \subset \mathbb{X}$, the integrands must be equal almost everywhere, and the result follows:

$$K(x^{(t-1)}, x^{(t)}) = \alpha(x^{(t)} \mid x^{(t-1)}) q(x^{(t)} \mid x^{(t-1)}) + (1 - a(x^{(t-1)})) \delta_{x^{(t-1)}}(x^{(t)})$$

□

C Proof of Detailed Balance

Detailed balance is achieved for the Metropolis-Hastings algorithm when:

$$\pi(x^{(t-1)}) q(x^{(t)} \mid x^{(t-1)}) \alpha(x^{(t)} \mid x^{(t-1)}) = \pi(x^{(t)}) q(x^{(t-1)} \mid x^{(t)}) \alpha(x^{(t-1)} \mid x^{(t)}).$$

Proof. We can show this by manipulating the left-hand side and recovering the right-hand side. Expanding the left-hand side using the definition of α :

$$\begin{aligned}
& \pi(x^{t-1}) q(x^t \mid x^{t-1}) \alpha(x^t \mid x^{t-1}) \\
&= \pi(x^{t-1}) q(x^t \mid x^{t-1}) \min \left(1, \frac{\pi(x^t) q(x^{t-1} \mid x^t)}{\pi(x^{t-1}) q(x^t \mid x^{t-1})} \right) \\
&= \min (\pi(x^{t-1}) q(x^t \mid x^{t-1}), \pi(x^t) q(x^{t-1} \mid x^t)) \\
&= \pi(x^t) q(x^{t-1} \mid x^t) \min \left(1, \frac{\pi(x^{t-1}) q(x^t \mid x^{t-1})}{\pi(x^t) q(x^{t-1} \mid x^t)} \right) \\
&= \pi(x^t) q(x^{t-1} \mid x^t) \alpha(x^{t-1} \mid x^t).
\end{aligned}$$

We can now verify detailed balance for the full kernel. Expanding $\pi(x^{(t-1)})K(x^{(t-1)}, x^{(t)})$ using the previous proof:

$$\begin{aligned}
& \pi(x^{(t-1)})K(x^{(t-1)}, x^{(t)}) \\
&= \pi(x^{(t-1)}) q(x^{(t)} \mid x^{(t-1)}) \alpha(x^{(t)} \mid x^{(t-1)}) \\
&+ \pi(x^{(t-1)})(1 - a(x^{(t-1)})) \delta_{x^{(t-1)}}(x^{(t)}) \\
&= \pi(x^{(t)}) q(x^{(t-1)} \mid x^{(t)}) \alpha(x^{(t-1)} \mid x^{(t)}) \\
&+ \pi(x^{(t)})(1 - a(x^{(t)})) \delta_{x^{(t)}}(x^{(t-1)}) \\
&= \pi(x^{(t)})K(x^{(t)}, x^{(t-1)}).
\end{aligned}$$

For the second term, note that $\delta_{x^{(t-1)}}(x^{(t)})$ is zero unless $x^{(t)} = x^{(t-1)}$, in which case $\pi(x^{(t-1)}) = \pi(x^{(t)})$ and $a(x^{(t-1)}) = a(x^{(t)})$, so the substitution holds. $\square$

References

- Michael Betancourt. A conceptual introduction to hamiltonian monte carlo, 2018. URL <https://arxiv.org/abs/1701.02434>.
- Christopher Bishop. *Pattern Recognition and Machine Learning*. Information science and statistics. Springer (India) Private Limited, 2006. ISBN 9788132209065. URL <https://books.google.co.uk/books?id=HL4HrgEACAAJ>.
- Rohitash Chandra and Joshua Simmons. Bayesian neural networks via mcmc: A python-based tutorial. *IEEE Access*, 12:70519–70549, 2024. ISSN 2169-3536. doi: 10.1109/access.2024.3401234. URL <http://dx.doi.org/10.1109/ACCESS.2024.3401234>.
- Tianqi Chen, Emily B. Fox, and Carlos Guestrin. Stochastic gradient hamiltonian monte carlo, 2014. URL <https://arxiv.org/abs/1402.4102>.
- George Deligiannidis. Advanced simulation methods: Chapter 7 - hamiltonian monte carlo. Lecture notes, Department of Statistics, University of Oxford, a. URL <https://www.stats.ox.ac.uk/~deligian/pdf/sc5/notes/notes7.pdf>.
- George Deligiannidis. Advanced simulation methods: Chapter 6 – the metropolis–hastings algorithm. Lecture notes, Department of Statistics, University of Oxford, b. URL <https://www.stats.ox.ac.uk/~deligian/pdf/sc5/notes/notes6.pdf>.
- Elias Dritsas and Maria Trigka. Exploring the intersection of machine learning and big data: A survey. *Machine Learning and Knowledge Extraction*, 7(1), 2025. ISSN 2504-4990. doi: 10.3390/make7010013. URL <https://www.mdpi.com/2504-4990/7/1/13>.
- Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo, 2011. URL <https://arxiv.org/abs/1111.4246>.
- Yann LeCun and Corinna Cortes. The mnist database of handwritten digits.
- Yi-An Ma, Tianqi Chen, and Emily B. Fox. A complete recipe for stochastic gradient mcmc, 2015. URL <https://arxiv.org/abs/1506.04696>.
- Kevin Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL <http://probml.github.io/book1>.
- Radford Neal. Mcmc using hamiltonian dynamics. *Handbook of markov chain monte carlo*, pages 47–95, 2012.
- James Norris. Markov chains. URL <https://www.statslab.cam.ac.uk/~rrw1/markov/M.pdf>. Lecture Notes.
- Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- Issei Sato and Hiroshi Nakagawa. Approximation analysis of stochastic gradient langevin dynamics by using fokker-planck equation and ito process. In Eric P. Xing and Tony Jebara, editors, *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 982–990, Beijing, China, 22–24 Jun 2014. PMLR.
- Max Welling and Yee Whye Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th International Conference on International Conference on Machine Learning*, ICML’11, pages 681–688, Madison, WI, USA, 2011. Omnipress. ISBN 9781450306195.